DOCUMENT 01

DOCUMENT 01

Product Definition & Complete User Journey

1. Product Definition

A **personal AI-assisted exam preparation application** designed for a candidate preparing for an Uttarakhand accountant/accounting-related government examination with approximately one month remaining.

The application has **two core modules**:

A. Revision Scheduler

Helps the candidate:

- Add and organize the complete syllabus/topics
- Define available study time
- Set the exam date
- Generate a revision plan
- Track every revision
- Know exactly what to revise each day
- Dynamically prioritize topics based on revision history and performance

B. Mock Test Dashboard

Helps the candidate:

- Upload MCQ questions as images
- Convert them into structured interactive questions using OpenAI
- Configure mock tests
- Attempt questions in a timed test environment
- Track answer and time spent per question
- Save every test attempt
- Analyse performance and weak areas

These modules remain separately usable but share **topic and performance data**.

---

2. Core Product Goal

The application should solve one simple problem:

> **The candidate should never have to decide what to revise next or struggle to understand where preparation is weak.**

Every day, the system should provide clarity:

What should I revise today?

What have I already revised?

What is overdue?

Which topics am I weak in?

How am I performing in tests?

What should receive more attention before the exam?

---

3. Primary User

Initially, this application is designed for **one primary candidate**, not as a multi-user commercial exam platform.

This affects our product decisions:

- No complex social features
- No leaderboards
- No student communities
- No teacher/admin ecosystem
- No subscription/payment system
- No unnecessary multi-exam complexity

The focus is **personal preparation efficiency**.

---

4. Application Structure

EXAM PREPARATION APP
│
├── DASHBOARD
│
├── REVISION SCHEDULER
│ ├── Subjects
│ ├── Chapters
│ ├── Topics
│ ├── Study Availability
│ ├── Daily Revision Plan
│ ├── Revision Tracking
│ └── Revision History
│
├── MOCK TESTS
│ ├── Upload MCQs
│ ├── AI Processing
│ ├── Review Questions
│ ├── Question Bank
│ ├── Create Test
│ ├── Attempt Test
│ └── Test History
│
└── ANALYTICS
├── Revision Progress
├── Test Performance
├── Topic Performance
└── Weak Areas

---

5. First-Time User Journey

Step 1 — Login

The candidate enters the application.

Since this is initially a personal tool, authentication should remain simple.

After login, a first-time user enters onboarding.

Step 2 — Exam Setup

The user configures:

- Exam name
- Exam date

Example:

> Uttarakhand Accountant Examination
> Exam Date: [Selected Date]

The exam date becomes the central deadline for the revision scheduler.

Step 3 — Add Study Availability

The user defines when she can study.

For example:

Day	Available Time
Monday	3 hours
Tuesday	3 hours
Wednesday	3 hours
Thursday	3 hours
Friday	3 hours
Saturday	6 hours
Sunday	6 hours

This may eventually be configured as actual time slots, such as:

> Monday: 7 PM–10 PM

The scheduler uses this information to determine available revision capacity.

Step 4 — Add Syllabus Structure

The user manually creates her preparation structure:

Subject
↓
Chapter
↓
Topic

Example:

ACCOUNTING
│
├── Financial Accounting
│ ├── Journal Entries
│ ├── Ledger
│ └── Trial Balance
│
├── Final Accounts
│ ├── Trading Account
│ ├── Profit & Loss Account
│ └── Balance Sheet

The system should make adding topics quick because the candidate may have a large syllabus to enter.

At this stage, she can also provide basic information for a topic where needed:

- Estimated revision time
- Difficulty
- Importance/priority

These inputs will support scheduling.

Step 5 — Generate Revision Plan

Once:

- Exam date
- Study availability
- Topics

are available, the system generates the initial revision plan.

The user does **not** manually place every topic onto a calendar.

Instead:

Topics
+
Available Study Time
+
Remaining Days
+
Revision Requirements
↓
REVISION PLAN

The result is a structured plan distributed across the remaining time.

---

6. Main Dashboard

After onboarding, the dashboard becomes the application's primary entry point.

The dashboard should remain focused and answer only the most important questions.

Today's Revision

Example:

Today — September 5

1. Partnership Accounts
Second Revision
Estimated: 45 min

2. Depreciation
Third Revision
Estimated: 30 min

3. Trial Balance
Quick Revision
Estimated: 20 min

The candidate can begin and complete tasks directly from here.

Revision Progress

Simple overview:

> Topics revised: 42 / 68
>
> Today's progress: 2 / 4 tasks
>
> Revision completion: 72%

Exam Countdown

Example:

> **24 DAYS LEFT**

This should remain visible but not dominate the entire interface.

Mock Test Snapshot

Example:

> Last Test Score: 76%
>
> Strongest: Journal Entries
>
> Needs Attention: Partnership Accounts

A direct action:

Take Mock Test

---

7. Daily Revision User Journey

This is the application's most important recurring journey.

User opens the application

OPEN APP
↓
VIEW TODAY'S PLAN
↓
START TOPIC REVISION
↓
STUDY OUTSIDE/USING HER MATERIAL
↓
RETURN TO APP
↓
MARK REVISION COMPLETE

The app does not need to become a note-taking or content-learning platform.

She studies from her existing books/material.

The application's role is to manage:

> **What to revise → When to revise → Whether it was revised → When it should come again**

Completing a Revision

When she completes a revision task, the system records:

- Topic
- Revision cycle/number
- Date
- Completion status

The scheduler then uses this completion to maintain future revision planning.

If required in the interface, she may also indicate a simple confidence level such as:

- Good
- Average
- Weak

This should remain optional and fast.

---

8. Missed Revision Journey

If the user does not complete today's planned tasks, the system must not simply delete them.

Example:

September 8
Partnership Accounts — NOT COMPLETED

The system marks it as overdue.

On the next scheduling cycle:

OVERDUE REVISION
+
UPCOMING PLAN
↓
REPRIORITIZE AVAILABLE TIME

The topic receives appropriate priority and is rescheduled according to the remaining capacity before the exam.

The user should not have to manually repair the entire schedule.

---

9. Mock Test User Journey

The second major workflow starts with MCQ preparation.

Step 1 — Upload Questions

The user creates or selects a question set.

Example:

Accounting MCQs — Set 01

She uploads images containing MCQs from her preparation books/material.

Potentially:

- One image
- Multiple images
- Multiple pages

Step 2 — AI Processes Questions

The backend sends the uploaded content for AI processing.

The system extracts:

Question
Option A
Option B
Option C
Option D
Correct Answer

Where possible, questions are also associated with a topic.

The result is **not immediately treated as perfect**.

Step 3 — Review Extracted Questions

The user sees the structured questions.

Example:

Question 01

Which of the following...

A. XXXXX
B. XXXXX
C. XXXXX
D. XXXXX

Correct Answer: B

She can correct extraction mistakes before the questions enter the question bank/test.

This review step is important because book images and scanned questions may contain extraction errors.

---

10. Create Mock Test

The user selects questions and configures the test.

Core configuration:

- Test name
- Number of questions
- Overall time limit
- Marks per question
- Negative marking, if applicable

Example:

Accounting Mock Test 01

50 Questions
60 Minutes
+1 Correct
-0.25 Incorrect

Then:

Start Test

---

11. Test Attempt Journey

The user enters a focused test interface.

QUESTION 11 OF 50

Which of the following is...

A. Option
B. Option
C. Option

D. Option

[Save & Next]

TIME REMAINING: 42:16

The system records:

- Answer selected
- Time spent on the question
- Whether the answer changes
- Skipped/unanswered state
- Mark for review status, if enabled

The user can navigate between questions.

Core actions:

- Previous
- Save & Next
- Mark for Review
- Question Navigator
- Submit Test

---

12. Test Completion

The test ends when:

A. User submits manually

or

B. Overall test time reaches zero

The system then calculates and saves the attempt.

The attempt should never disappear after the result is shown.

---

13. Test Results Journey

The candidate receives a result summary.

Overall

Score: 38 / 50
Accuracy: 76%
Time Taken: 47m 23s

Correct: 38
Incorrect: 8
Skipped: 4

Then performance can be viewed by subject/topic where topic mapping exists.

Example:

Topic	Performance
Journal Entries	90%
Ledger	82%
Depreciation	54%
Partnership Accounts	38%

The user can then review:

- Correct questions
- Incorrect questions
- Skipped questions
- Her selected answer
- Correct answer
- Time spent

---

14. Connection Between Both Modules

This connection is essential but should remain simple.

The workflow is:

REVISION
↓
MOCK TEST
↓
PERFORMANCE
↓
WEAK TOPICS IDENTIFIED
↓
REVISION PRIORITY INCREASED
↓
NEXT REVISION PLAN

Example:

She has revised **Partnership Accounts**, but performs poorly in multiple questions from that topic.

The system can treat the topic as needing greater attention during future planning.

This does **not** mean mock tests automatically take complete control over the schedule.

Test performance is one important input into revision priority.

---

15. Complete Screen List for V1

Onboarding

1. Login
2. Exam Setup
3. Study Availability
4. Add Subjects/Topics
5. Generate Initial Plan

Dashboard

6. Main Dashboard

Revision

7. Today's Revision Plan
8. Topics Overview
9. Topic Detail
10. Schedule/Calendar
11. Revision History
12. Study Availability Settings

Questions

13. Question Set List
14. Upload Questions
15. Processing Status
16. Review Extracted Questions
17. Question Bank

Mock Tests

18. Create Test
19. Test Configuration
20. Test Instructions
21. Test Runner
22. Question Navigator
23. Submit Confirmation

Results & Analytics

24. Test Result
25. Question Review
26. Test History
27. Performance Analytics

---

16. MVP Features

These are the features we are actually committing to.

Revision

- Exam date
- Study availability

- Subject/chapter/topic management
- Estimated revision duration
- Initial revision plan generation
- Daily revision tasks
- Mark complete
- Revision history
- Missed task handling
- Dynamic rescheduling

Questions

- Multiple image upload
- AI extraction
- Structured MCQ creation
- Manual review/editing
- Question storage
- Topic association

Mock Tests

- Create question set/test
- Overall timer
- Per-question time tracking
- Answer selection
- Previous/next navigation
- Question status tracking
- Mark for review
- Manual submission
- Auto-submit
- Attempt storage

Analytics

- Score
- Accuracy
- Correct/incorrect/skipped
- Total time
- Time per question
- Topic-wise performance
- Test history
- Weak-topic identification

---

17. Explicitly Outside V1

To protect the timeline and scope, we **do not build these initially**:

- Video lessons
- AI tutor/chatbot
- Notes editor
- Flashcards
- Social/community features
- Leaderboards
- Teacher dashboard
- Multi-student classroom
- Payment/subscriptions
- Native Android/iOS app
- Complex gamification
- Public question marketplace
- Automatic web research/content generation

The application remains focused on:

> **Revision Management + MCQ Testing + Performance Feedback**

---

18. Product Success Criteria

The application is successful if, every day, the candidate can open it and immediately understand:

1.

What should I revise today?

2.

What have I completed and what is pending?

3.

Which topics am I weak at?

4.

How am I performing in mock tests?

5.

What needs more attention before the exam?

If the system reliably answers those five questions, the product is doing its job.

DOCUMENT 01 STATUS: PRODUCT SCOPE LOCKED

Next, **Document 02 — Revision Scheduler & Intelligence Logic** will define the actual brain behind the revision planner: exactly how topics are prioritized, revision cycles are created, available time is allocated, missed tasks are handled, and the plan adapts as the exam approaches.

DOCUMENT 02

DOCUMENT 02

Revision Scheduler & Intelligence Logic

1. Purpose

This document defines the **brain of the Revision Scheduler**.

Its job is simple:

> Take all topics, the time available before the exam, and the candidate's actual progress—and continuously decide what should be revised and when.

This is **not an AI-generated calendar every time**. The core scheduling must be deterministic and reliable. AI can later assist with insights, but the actual scheduling engine should follow clear rules.

---

2. Core Inputs

The scheduler works from five main inputs:

EXAM DATE
+
TOPICS
+
AVAILABLE STUDY TIME
+
REVISION HISTORY
+
MOCK TEST PERFORMANCE
↓
DYNAMIC REVISION PLAN

A. Exam Date

The absolute deadline.

The system continuously knows:

Days remaining until exam = Exam Date − Current Date

As the exam gets closer, scheduling behaviour changes accordingly.

B. Topics

Every topic belongs to:

Subject → Chapter → Topic

Each topic should have:

- Name
- Subject
- Chapter
- Estimated revision duration
- Difficulty
- Importance
- Current status
- Revision history
- Confidence level, where provided
- Performance data, where available

C. Study Availability

The user defines available study capacity.

For example:

Monday: 7 PM–10 PM
Tuesday: 7 PM–10 PM
Wednesday: 7 PM–10 PM
Saturday: 10 AM–1 PM, 6 PM–9 PM

The scheduler uses the actual available time, not an assumed number of hours.

D. Revision History

For every completed revision, the system knows:

- Topic
- Revision number
- Date completed
- Planned duration

- Actual completion status

E. Mock Test Performance

Where questions are mapped to topics, performance becomes an additional priority signal.

Poor performance should increase the chance of that topic receiving more revision attention.

---

3. Topic Status System

A topic should always have a clear preparation state.

For V1:

Not Started

The topic has not yet entered a completed revision cycle.

Scheduled

The topic has an upcoming revision task.

Due

The topic should be revised today.

Completed

The currently scheduled revision was completed.

Overdue

A scheduled revision was missed.

This is task status.

Separately, the system tracks the topic's overall revision state:

Not Revised
Revision 1 Completed
Revision 2 Completed
Revision 3 Completed

...

There is **no artificial maximum number of revisions**. The exam date and remaining capacity determine how many times a topic can realistically reappear.

---

4. Revision Cycles

The system uses repeated revision rather than a simple:

> Study once → Done forever.

A topic moves through revision cycles:

Revision 1
↓
Revision 2
↓
Revision 3
↓
Further Reinforcement
↓
Final Revision

For the one-month preparation window, the scheduler should aim to create multiple exposures wherever possible.

However, it must not blindly force every topic through exactly the same number of revisions.

For example:

Difficult / weak topic

May receive:

> Revision 1 → Revision 2 → Revision 3 → Revision 4 → Final Revision

Strong / easy topic

May receive:

> Revision 1 → Revision 2 → Final Revision

The available time and topic priority determine this.

5. Revision Priority

Every topic receives a **dynamic priority score**.

The score is recalculated regularly.

Conceptually:

PRIORITY =
Importance
+
Difficulty
+
Weak Performance
+
Overdue Status
+
Time Since Last Revision
+
Exam Urgency
+
Low Confidence

This should be implemented as a weighted score, not through AI judgment.

For example:

Priority Score =

Importance Weight
+ Difficulty Weight
+ Performance Weakness Weight
+ Revision Due Weight
+ Overdue Weight
+ Low Confidence Weight
+ Exam Urgency Weight

The exact numerical weights can be configured during development, but the product logic remains the same.

6. Importance and Difficulty

When adding a topic, the user can assign:

Importance

- High
- Medium
- Low

Difficulty

- Hard
- Medium
- Easy

These do not permanently dictate the schedule. They provide the scheduler's initial understanding.

For example:

> Partnership Accounts
> High Importance + Hard

will initially receive more attention than:

> Basic Definitions
> Low Importance + Easy

But later, real performance can modify that priority.

---

7. Confidence Signal

When marking a revision complete, the user may optionally answer:

> How confident do you feel about this topic?

- Strong
- Okay
- Weak

This should be deliberately quick.

The effect:

Strong

The topic can wait longer before its next revision.

Okay

Normal revision spacing.

Weak

The next revision is brought closer and its priority increases.

This is useful because mock tests do not immediately cover every topic.

---

8. Revision Spacing Logic

The system should avoid putting the same topic on consecutive days unless it is particularly weak or urgent.

The basic principle:

> Each successful revision increases the normal gap before the next one.

A starting model can be:

Revision	Normal Next Gap
Revision 1	2–3 days
Revision 2	4–5 days
Revision 3	6–7 days
Revision 4+	Based on remaining exam time

These are **base intervals**, not fixed rules.

The actual next date is adjusted based on:

- Difficulty

- Importance
- Confidence
- Weak mock-test performance
- Overdue revisions
- Remaining days until the exam

For example, a weak high-priority topic may return after two days even after Revision 3.

---

9. Initial Plan Generation

When the user completes setup, the scheduler should not immediately create a rigid month-long timetable that never changes.

Instead, it should:

Step 1

Calculate total available study capacity until the exam.

Step 2

Calculate total estimated time required for all topics.

Step 3

Check whether the full revision target fits within the remaining capacity.

Step 4

Assign topics across available time based on initial priority.

Step 5

Leave room for future revisions and rescheduling.

The key principle:

> **Do not fill 100% of available study time with the initial plan.**

Some capacity should remain available for:

- Missed revisions

- Weak topics
- Mock-test feedback
- Additional revision
- Final-week adjustments

---

10. Daily Plan Generation

Each day, the scheduler generates or updates the day's plan.

The priority order should broadly be:

Priority 1 — Overdue critical tasks

High-priority missed revisions.

Priority 2 — Topics genuinely due for revision

Topics whose next revision interval has arrived.

Priority 3 — Weak topics

Based on confidence and mock-test performance.

Priority 4 — New/unrevised topics

Topics that still need to enter the revision cycle.

Priority 5 — Reinforcement

Strong topics requiring another revision based on time remaining.

The system fills the available study slots accordingly.

Example:

AVAILABLE TODAY: 3 HOURS

45 min — Partnership Accounts
    Overdue + Weak

30 min — Depreciation
    Revision 3 Due

45 min — Cost Accounting
High Priority

30 min — Trial Balance
Weak Mock Test Performance

30 min — Quick Revision
Strong Topic Reinforcement

---

11. Capacity Rule

The scheduler must never create an impossible day.

If the user has:

3 hours available

the total scheduled estimated revision duration should not exceed that capacity.

A small buffer should ideally remain.

For example:

Available: 180 minutes

Target planned work: approximately 150–165 minutes

This prevents the plan from becoming stressful and constantly overdue.

The remaining buffer absorbs:

- Slightly longer revision sessions
- Breaks
- Unexpected difficulty

---

12. What Happens When a Task Is Completed?

When the user marks a revision task complete:

TOPIC COMPLETED
↓
Save Revision History
↓
Increment Revision Count
↓
Record Completion Date
↓
Apply Confidence Signal
↓
Calculate Next Revision Eligibility
↓
Future Schedule Updated

The next revision should not necessarily appear immediately.

It enters the scheduler's future candidate pool and is scheduled when appropriate.

---

13. What Happens When a Task Is Missed?

A missed task becomes overdue.

It should not simply disappear or create duplicate revision tasks.

The system should:

MISSED TASK
↓
Mark Overdue
↓
Increase Priority
↓
Check Next Available Capacity
↓
Reschedule

The rescheduling should consider:

- Topic importance
- How overdue it is

- Exam urgency
- Existing upcoming workload

A missed low-priority task may move slightly later.

A missed high-priority weak topic should be brought forward.

---

14. Dynamic Rescheduling

The schedule must be treated as **living**, not static.

It should update when:

- A revision is completed
- A revision is missed
- Confidence is recorded
- Study availability changes
- New topics are added
- Topics are removed
- Mock-test performance changes topic priority
- The exam gets closer

Example:

A user misses two days.

The application should not force the user to manually reorganize thirty future tasks.

Instead:

MISSED DAYS
↓
Detect Uncompleted Work
↓
Recalculate Remaining Capacity
↓
Prioritize Critical Topics
↓
Redistribute Future Revisions

The system must be realistic.

If there is insufficient time to complete everything exactly as initially planned, it should prioritize rather than create impossible schedules.

---

15. New Topics Added Later

The candidate may remember or discover additional topics after setup.

When a new topic is added:

NEW TOPIC
↓
Estimate Revision Duration
↓
Assign Initial Priority
↓
Check Remaining Capacity
↓
Insert Into Future Plan

The system should not blindly push everything else forward.

It recalculates the overall workload and places the new topic according to its priority.

---

16. Study Availability Changes

If she changes availability from:

> 3 hours on weekdays

to:

> 2 hours on weekdays

the scheduler recalculates remaining capacity.

Likewise, if additional time becomes available, the system can use that capacity for:

- Additional weak-topic revision
- More revision cycles

- Catch-up tasks

The exam date remains the fixed boundary.

---

17. Final Revision Mode

As the exam approaches, the system should gradually shift behaviour.

It should not introduce heavy new revision loads during the final days unless absolutely necessary.

The general progression:

Early Period

Focus on coverage + repeated revision.

Middle Period

Focus on reinforcement + weak areas.

Final Period

Focus on:

- High-importance topics
- Frequently weak topics
- Recently incorrect MCQs
- Quick consolidated revision

The exact threshold for "final period" should be configurable, but for a one-month exam plan, roughly the last week is an appropriate starting point.

---

18. Final Week Behaviour

During the final week, the scheduler prioritizes:

HIGH IMPORTANCE
+

WEAK PERFORMANCE
+
FREQUENTLY MISSED/OVERDUE
+
LONG TIME SINCE REVISION

Strong, recently revised, low-importance topics should receive less time.

The goal changes from:

> Complete the ideal revision cycle.

to:

> Maximize readiness before the exam.

---

19. Mock Test Integration

Mock-test data should influence the scheduler through topic performance.

For example:

Partnership Accounts

Revision Confidence: Okay
Importance: High
Difficulty: Hard
Mock Test Accuracy: 38%

Its priority increases significantly.

Another example:

Trial Balance

Importance: Medium
Confidence: Strong
Mock Test Accuracy: 92%

Its priority can decrease temporarily.

This does not automatically mark topics complete or remove them from revision.

It simply changes **how urgently they compete for future study time**.

---

20. Scheduler Output

The primary output is not a complicated algorithm visible to the user.

The user should simply receive:

Today

Partnership Accounts
Revision 3 · 45 min

Depreciation
Revision 2 · 30 min

Cost Accounting
Revision 1 · 45 min

Trial Balance
Quick Revision · 20 min

The system internally manages the complexity.

The user only needs to know:

> **Do these things today.**

---

21. Core Scheduler Rules

These rules should remain fixed unless we intentionally change the product later:

1. The exam date is the final scheduling boundary.
2. Available study time determines daily capacity.
3. A topic is never considered permanently finished after one revision.
4. Revision intervals generally increase after successful revisions.

5. Weak topics receive more attention.
6. High-importance topics receive greater protection when time becomes limited.
7. Missed tasks are rescheduled, not deleted.
8. The system must never knowingly create impossible workloads.
9. Mock-test performance influences revision priority.
10. The plan dynamically adapts instead of remaining a rigid one-time calendar.
11. Final-week scheduling prioritizes exam readiness over ideal revision cycles.
12. Deterministic application logic controls scheduling; AI is not responsible for basic date/time decisions.

---

22. Final Scheduler Flow

USER ADDS TOPICS
+
EXAM DATE
+
STUDY AVAILABILITY
↓
CALCULATE TOTAL CAPACITY
↓
ESTIMATE REVISION WORKLOAD
↓
ASSIGN INITIAL PRIORITIES
↓
GENERATE INITIAL PLAN
↓
DAILY
↓
CHECK COMPLETIONS
CHECK MISSED TASKS
CHECK DUE REVISIONS
CHECK PERFORMANCE DATA
CHECK DAYS TO EXAM
↓
RECALCULATE PRIORITIES
↓
FIT TASKS INTO AVAILABLE TIME
↓
GENERATE/UPDATE DAILY PLAN

DOCUMENT 02 STATUS: REVISION SCHEDULER LOGIC LOCKED

Next is **DOCUMENT 03 — Mock Test & MCQ Processing Engine**, covering the complete pipeline from **uploading book images → OpenAI extraction → review/correction → question bank → test configuration → timed test runner → saved attempt**.

DOCUMENT 03

DOCUMENT 03

Mock Test & MCQ Processing Engine

1. Purpose

This module converts existing MCQs from the candidate's study material into a proper **interactive mock test system**.

The complete flow is:

MCQ IMAGES
↓
UPLOAD
↓
OPENAI EXTRACTION
↓
STRUCTURED QUESTIONS
↓
USER REVIEW & CORRECTION
↓
QUESTION BANK
↓
CREATE MOCK TEST
↓
TIMED TEST RUNNER
↓
SUBMISSION
↓
SAVE ATTEMPT + RESULTS

The objective is simple:

> She should be able to take MCQs available in books/images and attempt them like a real digital mock examination.

---

2. Question Source

For V1, the primary input is:

- Images of book pages
- Screenshots containing MCQs
- Photos containing MCQs

A single upload session may contain:

- 1 image
- Multiple images
- Multiple pages containing multiple questions

The system processes them as a **Question Set**.

Example:

> Question Set: Accounting Practice Set 01
> Uploaded Images: 12
> Questions Extracted: 48

---

3. Upload Flow

The user enters:

Question Sets

She can:

- View existing question sets
- Create a new question set
- Upload more images to an existing set

When creating a new set:

Set Name:
[Accounting Practice Set 01]

Optional Subject:
[Accounting]

Optional Topic:
[Select Topic]

[Upload Images]

Then she selects multiple images.

The system should show upload progress.

Uploading 8 of 12 images...

Once uploaded, processing begins.

---

4. Image Processing Pipeline

The images should not directly become test questions.

They pass through this pipeline:

IMAGE UPLOAD
↓
STORE ORIGINAL IMAGE
↓
SEND IMAGE FOR AI PROCESSING
↓
EXTRACT MCQs
↓
RETURN STRUCTURED DATA
↓
VALIDATE RESPONSE
↓
SAVE AS DRAFT QUESTIONS
↓
USER REVIEW
↓
APPROVED QUESTIONS

Original images should remain stored so the user can refer back to the source if an extracted question is incorrect.

---

5. OpenAI Extraction

OpenAI Vision is used to understand the uploaded image.

The system asks for structured extraction of each MCQ found.

For each question, the desired structure is conceptually:

Question Number
Question Text

Option A
Option B
Option C
Option D

Correct Answer
Explanation (only if present/available)

The system should support questions where:

- There are four options
- There are more or fewer options if the source contains them
- Question numbering is missing
- Multiple questions exist on one image
- Questions continue across images

The extraction pipeline must preserve the original wording as accurately as possible.

---

6. Correct Answer Handling

This is an important rule.

The system should **not blindly invent a correct answer** if it is not available in the uploaded material.

There are two cases:

Case A — Correct answer is visible/provided

The extracted answer is saved and marked as sourced from the uploaded content.

Case B — Correct answer is not present

The question is extracted without assuming an answer.

During review, the user can manually set the correct answer.

AI may later be used to suggest an answer, but for V1, **suggested answers should not silently become official answers**.

Every test question must have a confirmed correct answer before it can be used in scored testing.

---

7. Extraction Confidence & Validation

AI output can contain errors.

Before questions are approved, the system checks basic validity.

For example:

Valid question requires:

- Question text exists
- At least two options exist
- Options are distinguishable
- Correct answer, if required for testing, is valid

Potential problems:

Question text incomplete
Option C missing
Image text unclear
Correct answer not available

Such questions are flagged for review.

The system should not discard uncertain questions automatically.

They remain as drafts requiring user attention.

---

8. Review & Correction Screen

This screen is a mandatory quality-control step.

The user sees each extracted question in editable form.

Example:

Question 12

What is the meaning of depreciation?

A. XXXXX
B. XXXXX
C. XXXXX
D. XXXXX

Correct Answer: **B**

Actions:

- Edit question
- Edit options
- Add/delete option
- Set/change correct answer
- Assign subject
- Assign topic
- Delete question
- View original source image
- Approve question

The goal is not to make her manually rewrite every question.

Normally, she should only correct extraction mistakes.

---

9. Topic Association

Questions should be associated with the existing topic hierarchy wherever possible.

Subject
↓

Chapter
↓
Topic

Example:

Accounting
↓
Final Accounts
↓
Depreciation

Topic association is important because it enables later analytics.

The user can manually select a topic.

OpenAI may also suggest a topic based on the question, but the final saved mapping must use topics that already exist in the application's syllabus structure.

The system should not create random duplicate topics.

---

10. Question Status

Each question should have a clear lifecycle.

DRAFT
↓
NEEDS REVIEW
↓
APPROVED
↓
AVAILABLE FOR TESTS

Questions can also be:

- Rejected
- Deleted
- Edited after approval

Only approved questions with a confirmed answer are available for scored mock tests.

---

11. Question Bank

The Question Bank is the central repository of approved questions.

The user should be able to filter by:

- Question Set
- Subject
- Chapter
- Topic

Basic question information:

Question
Topic
Question Set
Correct Answer
Status

The user can:

- View questions
- Search questions
- Edit questions
- Delete questions
- Select questions for a test

No unnecessary advanced question-bank features are required for V1.

---

12. Create Mock Test

The user creates a test from the approved question bank.

Example:

> Create New Test

Configuration:

Test Name

Accounting Mock Test 01

Questions

Select manually or choose from a Question Set.

Number of Questions

50

Overall Time Limit

60 minutes

Marks Per Correct Answer

+1

Negative Marking

0 / -0.25 / Custom

For V1, the test should use an **overall test timer**.

The system still tracks time spent per question internally.

---

13. Test Configuration Rules

Before starting a test, the system validates:

- At least one question exists
- Every selected question has a correct answer
- Time limit is valid
- Marks configuration is valid

The system then creates a fixed test definition.

Once the candidate starts an attempt, changes to the question bank should **not alter that historical test attempt**.

The attempt uses its own stored snapshot of:

- Questions
- Options

- Correct answers
- Marks configuration

This ensures old results remain accurate even if questions are later edited.

---

14. Test Instructions Screen

Before the test begins, show:

Accounting Mock Test 01

- Total Questions: 50
- Duration: 60 Minutes
- Marks: +1 for correct
- Negative Marking: -0.25 for incorrect

Then:

START TEST

The timer should begin only after the user explicitly starts the test.

---

15. Test Runner

This is the core examination interface.

Example:

Question 11 of 50

Time Remaining: 42:16

Which of the following...

A. Option
B. Option
C. Option
D. Option

[Previous] [Save & Next]

The screen should remain focused.

No unnecessary dashboard navigation during an active test.

---

16. Question Navigation

The candidate can:

- Select an answer
- Save & Next
- Go Previous
- Jump to another question using the navigator
- Mark a question for review
- Submit the test

The question navigator should clearly show question states.

For example:

1 ✓ Answered
2 ✓ Answered
3 ● Current
4 ○ Not Answered
5 ★ Marked for Review

The visual implementation will be defined later, but these states are required.

---

17. Answer Saving

Every answer interaction should be saved immediately.

The system records:

- Question ID
- Selected option

- First answer timestamp
- Latest answer timestamp
- Number of answer changes
- Current answer status

This prevents unnecessary loss if the user:

- Refreshes the page
- Accidentally closes the browser
- Experiences a temporary interruption

---

18. Per-Question Time Tracking

Although the test has an overall timer, the system tracks time spent on each question.

Conceptually:

User opens Question 1
↓
Start tracking active time
↓
Moves to Question 2
↓
Save time spent on Question 1

If she returns to Question 1 later, additional active time is added.

Final data might be:

Question 1: 42 seconds
Question 2: 1 minute 18 seconds
Question 3: 19 seconds
Question 4: 2 minutes 05 seconds

This data is used later for performance analysis.

The system should track **active question-viewing time**, rather than assuming every second of total test time belongs to the currently displayed question when the app is inactive.

---

19. Question States During Test

Each question has one of the following practical states:

Not Visited

The user has not opened it.

Visited but Unanswered

The user viewed it but has not selected an answer.

Answered

An answer is currently selected.

Marked for Review

The user wants to revisit it.

Answered + Marked for Review

The user selected an answer but wants to check it later.

These states are useful for navigation and final review.

---

20. Mark for Review

Mark for Review does not mean the question is unanswered.

It is simply a user flag.

For example:

> "I'm unsure about this. Come back later."

The question can still have an answer selected.

Before final submission, the system may show:

> You have 4 questions marked for review.

But it should not prevent submission.

---

21. Test Timer

The overall timer begins at test start.

60:00
↓
59:59
↓
59:58

The timer continues until:

- The user submits, or
- Time reaches zero

The test should use a server-aware timestamp/reference so that refreshing the page cannot reset the timer.

The client display is only a representation of the actual attempt timing.

---

22. Interruption & Resume

If the candidate refreshes or temporarily loses access:

1. The current answers remain saved.
2. The original test end time remains unchanged.
3. On return, the system calculates the actual remaining time.
4. The candidate resumes from the test.

Example:

Test started: 10:00 AM
Duration: 60 minutes
Refresh: 10:25 AM

Remaining: 35 minutes

The timer does not restart.

---

23. Manual Submission

When the candidate clicks:

Submit Test

the system should show a confirmation summary.

Example:

Answered: 43 / 50
Unanswered: 4
Marked for Review: 3

[Go Back] [Submit Test]

The user can return to the test or confirm submission.

---

24. Auto-Submit

When the timer reaches zero:

TIME EXPIRED
↓
STOP TEST
↓
SAVE FINAL ANSWERS
↓
CALCULATE RESULT
↓
CREATE RESULT
↓
SHOW RESULT PAGE

No further answer modifications are allowed.

The final attempt becomes read-only.

---

25. Scoring Logic

After submission:

FOR EACH QUESTION:

IF Answer = Correct
    Add Correct Marks

IF Answer = Incorrect
    Apply Negative Marks

IF Unanswered
    Add 0

Example:

Correct: 38 × 1   = 38
Incorrect: 8 × -0.25 = -2
Unanswered: 4   = 0

Final Score = 36 / 50

Accuracy should be separately calculated.

For V1:

    Accuracy = Correct Answers ÷ Attempted Questions × 100

This avoids treating skipped questions as incorrect when analysing answer accuracy.

---

26. Attempt Storage

Once submitted, a permanent attempt record is created.

The system stores:

Test-level data

- Test name
- Attempt start time
- Submission time
- Total duration
- Time actually used
- Score
- Accuracy

Question-level data

- Question snapshot
- Selected answer
- Correct answer snapshot
- Correct/incorrect/skipped status
- Time spent
- Marked for review status

Historical attempts should remain unchanged.

---

27. Test Result Output

Immediately after submission:

Overall Result

Score: 36 / 50
Accuracy: 82.6%

Correct: 38
Incorrect: 8
Skipped: 4

Time Used: 47m 23s

The candidate can then move into detailed analysis.

Core actions:

- Review Questions
- View Topic Performance
- View Test History

- Return to Dashboard

The deeper analytics logic will be defined in Document 04.

---

28. Error Handling Rules

The system must handle:

Image processing fails

Show failed status and allow retry.

Some images succeed, some fail

Keep successful results; retry only failed images.

Extraction is incomplete

Save the extracted draft and flag it for review.

Correct answer missing

Question cannot enter a scored test until confirmed.

User leaves during processing

Processing status remains stored and can continue/check later.

Test connection interruption

Answers already saved remain available.

Timer expires during interruption

On return, the system detects that the end time has passed and submits the attempt.

---

29. Core Product Rules

These rules are locked for this module:

1. Original uploaded MCQ images are preserved.
2. AI extraction creates draft data, not unquestioned final data.
3. The user can review and correct extracted questions.
4. AI must not silently invent official correct answers.
5. Every scored test question requires a confirmed correct answer.
6. Approved questions enter the Question Bank.
7. Tests use an overall configurable timer.
8. Time spent per question is also tracked.
9. Answers are saved during the test.
10. Refreshing cannot reset the test timer.
11. Completed attempts are permanently stored.
12. Historical attempts use immutable question/test snapshots.
13. Mock-test results can later influence revision priority.

---

30. Final Module Flow

CREATE QUESTION SET
↓
UPLOAD MCQ IMAGES
↓
STORE ORIGINALS
↓
OPENAI IMAGE PROCESSING
↓
EXTRACT QUESTIONS
↓
VALIDATE
↓
DRAFT / NEEDS REVIEW
↓
USER CORRECTION
↓
APPROVE QUESTIONS
↓
QUESTION BANK
↓
SELECT QUESTIONS
↓
CONFIGURE MOCK TEST

↓
START TEST
↓
ANSWER + TRACK TIME
↓
SAVE ANSWERS CONTINUOUSLY
↓
SUBMIT / AUTO-SUBMIT
↓
CALCULATE SCORE
↓
SAVE ATTEMPT
↓
RESULTS + ANALYTICS

DOCUMENT 03 STATUS: MOCK TEST & MCQ PROCESSING ENGINE LOCKED

Next is **DOCUMENT 04 — Analytics & Adaptive Learning System**. This will define exactly how the app converts revision history and mock-test attempts into **performance tracking, weak-topic detection, improvement trends, and revision-priority adjustments**.

DOCUMENT 04

DOCUMENT 04

Analytics & Adaptive Learning System

1. Purpose

This module turns raw preparation activity into useful decisions.

The application already knows:

- What topics exist
- What has been revised
- How often each topic was revised
- Revision confidence
- Mock-test answers
- Correct/incorrect/skipped questions
- Time spent per question
- Topic-wise question performance

Analytics should convert this into one clear outcome:

> **Show the candidate what is going well, what is weak, and what deserves more attention next.**

The system should not overwhelm her with unnecessary charts or statistics.

---

2. Core Analytics Flow

REVISION DATA
+
MOCK TEST DATA
↓
PROCESS PERFORMANCE
↓
TOPIC-WISE UNDERSTANDING
↓
IDENTIFY STRENGTHS & WEAKNESSES

↓
UPDATE REVISION PRIORITY
↓
GENERATE CLEAR INSIGHTS

---

3. Overall Performance Metrics

For every completed mock test, the system calculates:

Score

Based on the configured marking scheme.

Correct

Total correctly answered questions.

Incorrect

Total incorrectly answered questions.

Skipped

Total unanswered questions.

Attempted

Correct + Incorrect.

Accuracy

$$\frac{\text{Correct Answers}}{\text{Attempted Questions}} \times 100$$

Time Used

Actual duration between test start and completion, limited by the test rules.

Average Time Per Attempted Question

$$\frac{\text{Total Active Question Time}}{\text{}}$$

Attempted Questions

These become the core summary metrics.

---

4. Topic-Wise Performance

This is the most important analytical layer.

Because questions are associated with topics, every attempt contributes data to that topic.

Example:

Topic	Correct	Incorrect	Skipped	Accuracy
Journal Entries	18	2	1	90%
Ledger	10	3	2	76.9%
Depreciation	5	5	1	50%
Partnership Accounts	3	7	2	30%

The application should aggregate this data over multiple tests.

A single bad test should not permanently define a topic as weak.

---

5. Topic Performance Profile

Each topic should maintain a performance profile based on available data.

Conceptually:

TOPIC PERFORMANCE PROFILE

Accuracy
+
Number of Questions Attempted

+
Recent Performance
+
Performance Trend
+
Average Time Taken
+
Revision Confidence

This creates a more useful picture than looking at only one score.

For example:

Depreciation

- Overall Accuracy: 58%
- Recent Accuracy: 42%
- Questions Attempted: 34
- Average Time: High
- Last Revision Confidence: Weak

Result:

> **High Attention Required**

---

6. Weak Topic Detection

A topic can be identified as weak through multiple signals.

Signal 1 — Low Accuracy

Repeatedly incorrect answers indicate weak understanding.

Signal 2 — Slow Performance

The candidate may eventually answer correctly but take unusually long.

This suggests the topic is not yet strong enough for an exam environment.

Signal 3 — Recent Decline

For example:

Test 1: 80%
Test 2: 65%
Test 3: 48%

The recent pattern deserves attention.

Signal 4 — Low Revision Confidence

The candidate repeatedly marks:

Weak

after revising the topic.

Signal 5 — Combination

The strongest weak-topic signal is a combination of poor objective and subjective performance.

Example:

Low Accuracy
+
Slow Answers
+
Weak Confidence
↓
STRONG WEAK-TOPIC SIGNAL

---

7. Strong Topic Detection

A topic should not keep receiving the same level of attention forever if performance consistently shows strength.

A strong topic generally has:

- Consistently good accuracy
- Sufficient number of attempted questions
- Stable or improving recent performance
- Normal/fast answer time
- Strong or okay revision confidence

Example:

Trial Balance
Accuracy: 91%
Recent Performance: Stable
Revision Confidence: Strong

Result:

Lower Immediate Revision Priority

This does not remove the topic from future revision. It simply prevents strong topics from consuming time needed elsewhere.

---

8. Insufficient Data Rule

The system must distinguish between:

Weak topic

and

Not enough information yet.

For example:

A topic with only two attempted questions and one incorrect answer should not automatically be classified as weak.

Instead:

Limited Test Data

This prevents overreaction.

Topic analytics should become more reliable as more questions and revisions are recorded.

---

9. Recent Performance Matters More

Older performance should not permanently dominate the candidate's current status.

Example:

Older Tests: 40%, 45%, 50%
Recent Tests: 78%, 84%, 81%

The candidate is clearly improving.

The system should recognise recent improvement rather than continuing to treat the topic as equally weak.

Therefore, topic performance should give greater importance to recent attempts than very old attempts.

---

10. Performance Trend

Where enough historical data exists, the system identifies a simple trend:

Improving

Recent performance is meaningfully better.

Stable

No meaningful change.

Declining

Recent performance is meaningfully worse.

Example:

Depreciation

Test 1 → 45%
Test 2 → 58%
Test 3 → 72%

Trend: Improving ↑

Trend should remain a supporting signal, not replace actual accuracy.

---

11. Speed Analysis

Time matters in competitive exams.

For each question and topic, the system can analyse:

- Average time spent
- Slowest questions
- Faster-than-normal performance
- Accuracy versus speed

The important distinction:

Fast + Correct

Strong performance.

Slow + Correct

Knowledge exists, but speed may need improvement.

Fast + Incorrect

Possible careless guessing or weak understanding.

Slow + Incorrect

High-priority weakness.

Conceptually:

ACCURACY + SPEED
↓
PERFORMANCE QUALITY

---

12. Question-Level Analysis

After a test, the candidate should be able to review individual questions.

For each question:

- Question

- Her answer
- Correct answer
- Result
- Time spent

Question states:

Correct

No immediate problem.

Incorrect

A mistake requiring potential review.

Skipped

Knowledge gap, uncertainty, or time-management issue.

Marked for Review

Shows uncertainty during the test.

This information supports detailed self-review.

---

13. Repeated Mistake Detection

The system should identify when the candidate repeatedly struggles with the same topic.

Example:

Partnership Accounts

Test 1: 3/8 correct
Test 2: 2/7 correct
Test 3: 4/10 correct

This is stronger evidence than one bad test.

Result:

Repeated Weak Performance

The scheduler should give this topic increased attention.

For V1, this is tracked at the **topic level**. We do not need complex sub-concept error clustering yet.

---

14. Revision Progress Analytics

The system should also show preparation progress independent of mock tests.

Core metrics:

Topics Entered

Total topics in the syllabus.

Topics Revised at Least Once

Total Completed Revision Sessions

Topics Never Revised

Currently Due

Currently Overdue

Recent Revision Activity

Example:

68 Total Topics

54 Revised At Least Once
8 Never Revised
4 Due Today
2 Overdue

This tells the candidate whether coverage itself is becoming a problem.

---

15. Revision Effectiveness

The system can compare:

Revision activity versus later test performance.

Example:

Topic: Depreciation

Before Recent Revision:
Accuracy: 48%

After Recent Revision:
Accuracy: 72%

This indicates improvement.

Conversely:

Repeated Revisions
+
Continued Low Accuracy
↓
TOPIC STILL NEEDS ATTENTION

For V1, this should be presented as a simple insight when sufficient data exists—not as overly complicated scientific analysis.

---

16. Test History

The application maintains all previous mock-test attempts.

For each attempt:

- Test name
- Date
- Score
- Accuracy
- Time used

Example:

Test	Date	Score	Accuracy
Mock Test 01	Sep 5	36/50	76%
Mock Test 02	Sep 8	40/50	83%
Mock Test 03	Sep 12	38/50	79%

The candidate can open any previous attempt and review its detailed result.

Historical data is never overwritten.

---

17. Overall Improvement Tracking

Where multiple tests exist, the system can show simple preparation movement.

For example:

TEST PERFORMANCE

Mock 01 → 72%
Mock 02 → 76%
Mock 03 → 81%

Overall Trend: Improving

This should be interpreted carefully because different tests may have different difficulty levels.

Therefore:

> Test-to-test scores show progress signals, not absolute proof of improvement.

Topic-level trends remain more useful when similar topics are repeatedly tested.

---

18. Weakness Priority Levels

To keep the dashboard understandable, topics can be grouped into simple attention levels.

High Attention

Strong evidence of weakness.

Examples:

- Repeated low accuracy
- High importance + poor performance
- Weak confidence + poor results
- Slow and incorrect repeatedly

Needs Attention

Some weakness is visible, but not critical.

Stable

Performance is acceptable.

Strong

Consistently performing well.

Limited Data

Not enough test information yet.

These labels are easier for the candidate to understand than exposing raw internal priority scores.

---

19. Adaptive Revision Integration

Analytics directly feeds Document 02's scheduler.

The flow is:

MOCK TEST COMPLETED
↓
UPDATE TOPIC PERFORMANCE
↓
RECALCULATE WEAKNESS SIGNAL
↓

UPDATE TOPIC PRIORITY
↓
SCHEDULER REEVALUATES
↓
FUTURE REVISION PLAN ADAPTS

Example:

Before test

Depreciation
Priority: Medium

Test result

Accuracy: 42%
Average Time: Slow

After analysis

Depreciation
Attention: High
Revision Priority: Increased

The scheduler may then bring the next appropriate revision closer.

---

20. Important Adaptation Rule

The system should **not destroy the entire schedule after every test**.

Mock-test performance changes priorities, but the scheduler still considers:

- Exam date
- Existing due revisions
- Overdue work
- Study capacity
- Topic importance
- Remaining topics

So the correct behaviour is:

> **Adjust intelligently, not react chaotically.**

A single wrong answer should not rebuild the month.

Repeated or meaningful evidence should influence the plan more strongly.

---

21. Dashboard Analytics

The main dashboard should show only a concise snapshot.

Example:

Preparation

54 / 68 Topics Revised

Today

2 / 4 Revision Tasks Complete

Last Mock

76% Accuracy

High Attention

Partnership Accounts

Trend

Improving

The user can open detailed analytics separately.

---

22. Detailed Analytics Page

The dedicated Analytics area should contain:

Overall Performance

- Recent score
- Accuracy

- Time usage
- Overall trend

Topic Performance

- Accuracy
- Attention level
- Trend
- Question count

Weak Areas

Prioritized list of topics requiring attention.

Strong Areas

Topics performing consistently well.

Revision Progress

- Total coverage
- Revisions completed
- Due
- Overdue

Test History

Previous attempts with detailed access.

No additional analytics are required for V1.

---

23. AI-Generated Insights

OpenAI can optionally turn existing calculated data into concise human-readable observations.

For example:

> "Your overall accuracy is improving, but Partnership Accounts continues to show low accuracy across multiple attempts. Prioritize it in your next revision sessions."

Important rule:

AI receives the analytics and explains them.

AI does **not** independently calculate official scores, accuracy, or scheduling logic.

The application calculates the facts first.

APPLICATION DATA + CALCULATIONS
↓
AI INSIGHT
↓
HUMAN-READABLE OBSERVATION

This keeps analytics reliable and API usage controlled.

---

24. Core Analytics Rules

These rules are fixed for V1:

1. Every completed test updates relevant topic performance.
2. A single bad answer does not automatically define a weak topic.
3. Topics with insufficient data are labelled accordingly.
4. Recent performance matters more than old performance.
5. Accuracy and speed are analysed separately and together.
6. Repeated weak performance receives stronger attention.
7. Revision confidence contributes to the topic profile.
8. Strong topics receive reduced immediate priority, not permanent removal.
9. Mock-test results influence revision priority.
10. The scheduler adapts gradually and intelligently.
11. Historical test attempts remain unchanged.
12. AI may explain calculated insights but does not replace deterministic calculations.

---

25. Complete Adaptive Learning Loop

TODAY'S REVISION PLAN
↓
CANDIDATE REVISES TOPICS
↓
COMPLETION + CONFIDENCE RECORDED
↓

TAKES MOCK TEST
↓
ANSWERS + TIME DATA SAVED
↓
SCORE & PERFORMANCE CALCULATED
↓
TOPIC-WISE ANALYSIS
↓
WEAK / STABLE / STRONG IDENTIFICATION
↓
REVISION PRIORITIES UPDATED
↓
SCHEDULER ADAPTS FUTURE PLAN
↓
NEXT REVISION

This is the central intelligence loop of the whole application:

Revise → Test → Analyse → Adapt → Revise Again

DOCUMENT 04 STATUS: ANALYTICS & ADAPTIVE LEARNING SYSTEM LOCKED

Next: **DOCUMENT 05 — Complete Database & Data Architecture**, where we define the exact data entities and relationships for the entire application: users, exams, subjects, chapters, topics, availability, revision tasks, revision history, question sets, questions, tests, attempts, answers, and analytics data.

DOCUMENT 05

DOCUMENT 05

Complete Database & Data Architecture

1. Purpose

This document defines **what data the application stores, how entities connect, and what must remain historically preserved**.

The database should support the two core modules:

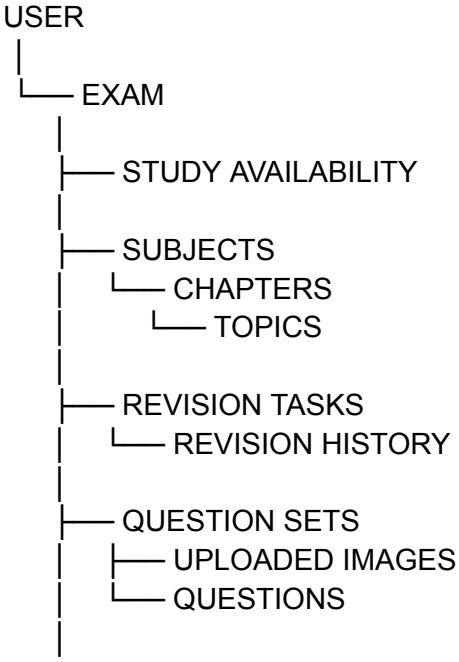
1. **Revision Scheduler**
2. **Mock Test System**

with Analytics using data generated by both.

For V1, a relational database structure is the right fit.

---

2. High-Level Data Structure



```
├── MOCK TESTS
│    └── TEST QUESTIONS
│
└── TEST ATTEMPTS
     └── ATTEMPT ANSWERS
```

---

3. User

Even though V1 is initially built for one candidate, the architecture should support users properly.

`users`

Core fields:

- `id`
- `name`
- `email`
- `created_at`
- `updated_at`

The user owns all exam preparation data.

---

4. Exam

The exam acts as the central preparation context.

`exams`

Fields:

- `id`
- `user_id`
- `name`
- `exam_date`
- `created_at`
- `updated_at`

Example:

Exam:
Uttarakhand Accountant Examination

Exam Date:
October XX, 2026

All subjects, topics, revision tasks and tests belong to an exam context.

---

5. Study Availability

The scheduler needs to know when the candidate can study.

`study_availability`

Fields:

- `id`
- `exam_id`
- `day_of_week`
- `start_time`
- `end_time`
- `created_at`
- `updated_at`

One day can have multiple slots.

Example:

Monday
7:00 PM–9:00 PM
9:30 PM–10:30 PM

Therefore, each slot should be a separate record.

This structure supports future schedule changes without redesigning the database.

---

6. Subject

`subjects`

Fields:

- `id`
- `exam_id`
- `name`
- `sort_order`
- `created_at`
- `updated_at`

Example:

Accounting
Cost Accounting
General Knowledge

---

7. Chapter

`chapters`

Fields:

- `id`
- `subject_id`
- `name`
- `sort_order`
- `created_at`
- `updated_at`

Relationship:

ONE SUBJECT
↓
MANY CHAPTERS

---

8. Topic

This is one of the application's most important entities.

`topics`

Fields:

- `id`
- `chapter_id`
- `name`
- `estimated_revision_minutes`
- `difficulty`
- `importance`
- `created_at`
- `updated_at`

Suggested values:

Difficulty

- `easy`
- `medium`
- `hard`

Importance

- `low`
- `medium`
- `high`

A topic should **not permanently store a simple "completed" boolean**, because it can be revised many times.

Its preparation state comes from revision tasks/history.

---

9. Topic Performance Profile

The system will repeatedly calculate analytics for a topic.

Some data should be stored as derived values for faster dashboard access.

`topic_performance_profiles`

Fields:

- `id`
- `topic_id`
- `questions_attempted`
- `correct_answers`
- `incorrect_answers`
- `skipped_answers`
- `accuracy`
- `average_time_seconds`
- `recent_accuracy`
- `performance_trend`
- `attention_level`
- `revision_priority_score`
- `updated_at`

Possible attention levels:

- `high_attention`
- `needs_attention`
- `stable`
- `strong`
- `limited_data`

This table is a **current performance summary**, not the source of truth.

The actual source remains historical test attempts and answers.

---

10. Revision Tasks

A revision task represents a specific scheduled revision.

Example:

Revise Depreciation
Revision 3
September 12
Estimated: 30 minutes

`revision_tasks`

Fields:

- `id`
- `exam_id`
- `topic_id`
- `revision_number`
- `scheduled_date`
- `scheduled_start_time` *(optional where slot planning is used)*
- `estimated_minutes`
- `status`
- `priority_score`
- `created_at`
- `updated_at`
- `completed_at`

Statuses:

- `scheduled`
- `due`
- `completed`
- `overdue`
- `cancelled`

A topic can have many revision tasks.

ONE TOPIC
↓
MANY REVISION TASKS

---

11. Revision History

Revision history should preserve completed activity.

`revision_history`

Fields:

- `id`
- `topic_id`
- `revision_task_id`
- `revision_number`
- `completed_at`
- `confidence`
- `estimated_minutes`
- `notes` *(optional and not required for UI emphasis)*
- `created_at`

Confidence:

- `strong`
- `okay`
- `weak`

This allows the scheduler to answer:

- When was this last revised?
- How many times?
- What confidence was reported?
- How long has it been since the last revision?

---

12. Question Sets

A Question Set groups uploaded MCQ material.

`question_sets`

Fields:

- `id`
- `exam_id`
- `name`
- `subject_id` *(optional)*

- `chapter_id` *(optional)*
- `topic_id` *(optional)*
- `created_at`
- `updated_at`

Example:

Accounting Practice Set 01

A set can contain many uploaded images and many extracted questions.

---

13. Uploaded Question Images

Original uploaded material must be preserved.

`question_source_images`

Fields:

- `id`
- `question_set_id`
- `storage_path`
- `original_file_name`
- `mime_type`
- `upload_status`
- `processing_status`
- `created_at`
- `processed_at`

Processing status:

- `uploaded`
- `processing`
- `completed`
- `failed`

The actual image file lives in cloud/file storage; the database stores its reference.

---

14. Questions

This is the central question-bank entity.

`questions`

Fields:

- `id`
- `exam_id`
- `question_set_id`
- `source_image_id` *(nullable where multiple images are involved)*
- `subject_id`
- `chapter_id`
- `topic_id`
- `question_text`
- `extraction_status`
- `approval_status`
- `correct_option_id` *(nullable until confirmed)*
- `created_at`
- `updated_at`

Extraction/approval states can remain simple:

- `draft`
- `needs_review`
- `approved`
- `rejected`

Only approved questions with a confirmed answer can enter scored tests.

---

15. Question Options

Options should be stored separately rather than as fixed `option_a`, `option_b`, etc.

This allows flexibility.

`question_options`

Fields:

- `id`
- `question_id`
- `option_label`
- `option_text`
- `sort_order`
- `created_at`
- `updated_at`

Example:

A → Straight Line Method
B → Written Down Value Method
C → ...
D → ...

Relationship:

ONE QUESTION
↓
MANY OPTIONS

The correct answer points to one option.

---

16. Mock Tests

A mock test is a reusable test definition.

`mock_tests`

Fields:

- `id`
- `exam_id`
- `name`
- `time_limit_minutes`
- `marks_per_correct`
- `negative_marks_per_incorrect`
- `status`

- `created_at`
- `updated_at`

Status:

- `draft`
- `ready`
- `archived`

---

17. Test Questions

A test should store its own question membership and order.

`mock_test_questions`

Fields:

- `id`
- `mock_test_id`
- `question_id`
- `sort_order`
- `created_at`

Relationship:

ONE MOCK TEST
↓
MANY TEST QUESTIONS

This allows a question to belong to multiple tests.

---

18. Test Attempts

Every time the candidate starts a test, a new attempt is created.

`test_attempts`

Fields:

- `id`
- `mock_test_id`
- `user_id`
- `started_at`
- `ends_at`
- `submitted_at`
- `status`
- `score`
- `correct_count`
- `incorrect_count`
- `skipped_count`
- `attempted_count`
- `accuracy`
- `total_time_seconds`
- `created_at`
- `updated_at`

Statuses:

- `in_progress`
- `submitted`
- `auto_submitted`
- `expired`

The `ends_at` timestamp is especially important.

It ensures the timer cannot be reset by refreshing the application.

---

19. Attempt Question Snapshots

This is critical for historical accuracy.

If a question is edited later, an old test result must not change.

Therefore, when an attempt begins, the system stores the relevant question data as a snapshot.

`attempt_questions`

Fields:

- `id`
- `test_attempt_id`
- `original_question_id`
- `topic_id`
- `question_text_snapshot`
- `options_snapshot`
- `correct_answer_snapshot`
- `sort_order`
- `created_at`

This means:

QUESTION BANK CHANGES LATER
↓
OLD ATTEMPT REMAINS EXACTLY AS IT WAS

---

20. Attempt Answers

This stores the candidate's actual interaction.

`attempt_answers`

Fields:

- `id`
- `attempt_question_id`
- `selected_option_id` *(or selected option reference within snapshot)*
- `answer_status`
- `is_marked_for_review`
- `first_answered_at`
- `last_answered_at`
- `answer_change_count`
- `time_spent_seconds`
- `is_correct`
- `created_at`
- `updated_at`

Practical answer statuses:

- `not_visited`
- `unanswered`
- `answered`
- `marked_for_review`
- `answered_marked_for_review`

This supports the complete test runner behaviour.

---

21. AI Processing Jobs

Image processing should not be treated as an instant guaranteed action.

The application needs to track processing work.

`ai_processing_jobs`

Fields:

- `id`
- `question_set_id`
- `source_image_id`
- `job_type`
- `status`
- `error_message`
- `started_at`
- `completed_at`
- `created_at`

For V1:

Job Type

- `mcq_extraction`

Status

- `queued`

- `processing`
- `completed`
- `failed`

This makes retries and partial failures manageable.

---

22. AI Extraction Metadata

The original question remains the main data.

Optional AI metadata can be stored separately.

For example:

`question_extraction_metadata`

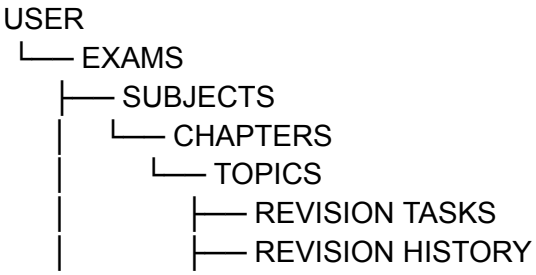
Fields:

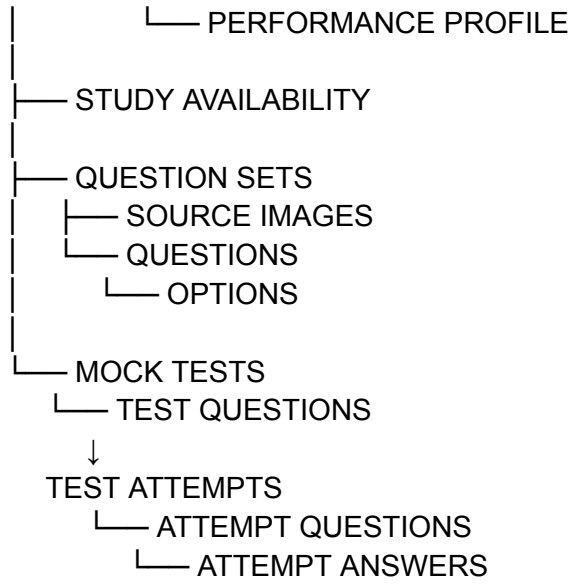
- `id`
- `question_id`
- `ai_confidence`
- `raw_extraction_reference`
- `requires_review`
- `created_at`

This helps identify uncertain extraction without mixing AI-processing details into the primary question record.

---

23. Key Relationships





---

24. Source of Truth vs Calculated Data

This distinction is important.

Source-of-truth data

These records should preserve actual events:

- Revision history
- Revision task completion
- Questions
- Options
- Test attempts
- Attempt answers
- Time spent
- Scores

Derived/calculated data

These can be recalculated:

- Topic accuracy
- Recent accuracy
- Performance trend
- Attention level

- Revision priority score
- Dashboard summaries

A cached `topic_performance_profile` is useful for speed, but the historical records remain authoritative.

---

25. Important Data Rules

Rule 1 — Do not overwrite history

Completed revisions remain recorded.

Submitted test attempts remain recorded.

Rule 2 — Attempts use snapshots

Historical results must survive future question edits.

Rule 3 — Topics are shared references

Questions should connect to existing topics rather than creating uncontrolled duplicate topic names.

Rule 4 — Files stay outside the database

Images belong in storage.

The database stores metadata and paths/references.

Rule 5 — Derived analytics can be rebuilt

If analytics logic improves later, profiles can be recalculated from original data.

Rule 6 — Deletion should be handled carefully

A question already used in a historical test should not destroy that attempt's record.

The preferred approach is to archive/deactivate where historical relationships matter.

---

26. Recommended Database Choice

For this product:

PostgreSQL

is the recommended primary database.

Why it fits:

- Clear relational structure
- Strong data consistency
- Good support for analytics queries
- Excellent compatibility with modern backend frameworks
- Supports JSON fields where useful, especially snapshots
- Easy to use with managed services

This keeps the architecture simple without requiring multiple databases.

---

27. Core V1 Database Entities

The essential V1 tables are:

users
exams

study_availability

subjects
chapters
topics

revision_tasks

revision_history

question_sets
question_source_images
questions
question_options

ai_processing_jobs
question_extraction_metadata

mock_tests
mock_test_questions

test_attempts
attempt_questions
attempt_answers

topic_performance_profiles

Nothing beyond these is required for the initial product scope.

---

28. Final Data Flow

USER SETUP
↓
EXAM + AVAILABILITY
↓
SUBJECTS → CHAPTERS → TOPICS
↓
REVISION TASKS
↓
REVISION HISTORY
↓
————————————————————
↓
UPLOAD MCQ IMAGES
↓
AI PROCESSING JOBS
↓
QUESTIONS + OPTIONS
↓

QUESTION BANK
↓
MOCK TEST
↓
TEST ATTEMPT
↓
ANSWER + TIME DATA
↓
TOPIC PERFORMANCE PROFILE
↓
REVISION PRIORITY UPDATE

DOCUMENT 05 STATUS: DATABASE & DATA ARCHITECTURE LOCKED

Next is **DOCUMENT 06 — OpenAI API & AI Architecture**, where we'll define exactly how the OpenAI API fits into this product, including **MCQ image extraction, structured outputs, validation, topic classification, performance insights, API cost control, error handling, and the strict boundary between AI decisions and deterministic system logic**.

DOCUMENT 06

DOCUMENT 06

OpenAI API & AI Architecture

1. Purpose

OpenAI should act as an **intelligence layer**, not as the core controller of the application.

The system itself must reliably control:

- Dates
- Timers
- Revision schedules
- Scores
- Accuracy
- Priority calculations
- Database updates

AI is used where understanding unstructured information is valuable.

The primary AI flow is:

UNSTRUCTURED MCQ IMAGES
↓
OPENAI
↓
STRUCTURED QUESTION DATA

Secondary AI use:

CALCULATED PERFORMANCE DATA
↓
OPENAI
↓
CLEAR HUMAN-READABLE INSIGHTS

---

2. V1 AI Responsibilities

For the first version, OpenAI is used for only these main jobs:

A. MCQ extraction from images

Read uploaded book pages/images and extract questions.

B. Question structuring

Convert extracted content into consistent structured data.

C. Topic classification

Suggest which existing syllabus topic a question belongs to.

D. Performance insights

Turn calculated analytics into concise, useful observations.

That is enough for V1.

We are **not** building a general AI tutor, chatbot, or autonomous scheduler.

---

3. Primary Architecture

USER
↓
UPLOAD IMAGE
↓
APPLICATION BACKEND
↓
STORE ORIGINAL IMAGE
↓
CREATE AI PROCESSING JOB
↓
OPENAI API
↓
STRUCTURED RESPONSE
↓
APPLICATION VALIDATION
↓
SAVE DRAFT QUESTIONS
↓

USER REVIEW
↓
APPROVED QUESTION BANK

The backend communicates with OpenAI.

The frontend should never expose the OpenAI API key.

---

4. MCQ Image Processing Flow

Step 1 — Upload

The user uploads one or multiple images.

The application:

1. Validates file type.
2. Stores the original image.
3. Creates a source-image record.
4. Creates an AI processing job.

Example:

Image: page_01.jpg
Status: Uploaded

AI Job:
Status: Queued

Step 2 — Process

A backend worker/process takes the job.

QUEUED
↓
PROCESSING
↓
OPENAI API REQUEST

The request contains the image and instructions to identify **all MCQs visible in it**.

The system must not assume:

One image = One question.

One page may contain ten questions.

---

5. Structured AI Output

The application should request structured output rather than free-form text.

Conceptually:

```
{
  "questions": [
    {
      "question_number": "1",
      "question_text": "...",
      "options": [
        {
          "label": "A",
          "text": "..."
        },
        {
          "label": "B",
          "text": "..."
        }
      ],
      "correct_answer": {
        "label": "B",
        "is_visible_in_source": true
      },
      "topic_suggestion": "Depreciation",
      "extraction_confidence": "high"
    }
  ]
}
```

The exact API schema will be implemented according to the OpenAI API's current supported structured-output format at build time.

The important product rule is:

AI returns machine-readable structured data that the backend validates before storing.

---

6. Extraction Instructions

The extraction prompt should clearly instruct the model to:

- Extract only visible MCQs.
- Preserve original wording as closely as possible.
- Preserve numerical values and formulas carefully.
- Extract every visible option.
- Maintain option order.
- Detect multiple questions on one image.
- Avoid merging separate questions.
- Flag incomplete questions.
- Flag unclear text.
- Never invent missing text.
- Never invent a correct answer.
- Explicitly distinguish between an answer visible in the source and an answer not present.

This is particularly important for accounting questions, where a small numerical extraction mistake can completely change the answer.

---

7. Correct Answer Rule

The AI must follow this strict boundary:

If the correct answer is clearly visible in the uploaded source:

Extract it.

If no correct answer is visible:

Return:

correct_answer: null

It must **not guess and silently save an answer as correct**.

Later, if we intentionally add an AI answer-suggestion feature, it must be clearly marked:

> AI Suggested — Requires Confirmation

But that is not required for V1.

---

8. Backend Validation

OpenAI output is not written directly into the production question bank.

The backend validates:

Question

- Does question text exist?

Options

- Are enough options present?
- Are labels/text valid?
- Are options duplicated?

Correct Answer

- If present, does it match one extracted option?

Topic

- Does the suggested topic correspond to an existing topic?

Completeness

- Is the extraction obviously incomplete?

The flow is:

OPENAI RESPONSE
↓
SCHEMA VALIDATION
↓
APPLICATION VALIDATION
↓

SAVE DRAFT
↓
USER REVIEW

---

9. Topic Classification

The model can help associate questions with the user's existing syllabus.

The system should provide the AI with a controlled list of available topics.

Example:

Available Topics:

1. Journal Entries
2. Ledger
3. Trial Balance
4. Depreciation
5. Partnership Accounts
6. Final Accounts

The AI should select from this list where a match exists.

It should not freely create:

> "Depreciation Accounting Methods Advanced"

when the actual system topic is simply:

> "Depreciation"

If no reliable match exists:

topic_id: null
requires_topic_review: true

The user then assigns the topic manually.

---

10. Image-by-Image Processing

For reliability, V1 should process images as individual jobs rather than sending an entire large set as one massive request.

Example:

12 UPLOADED IMAGES

Image 1 → Completed
Image 2 → Completed
Image 3 → Failed
Image 4 → Completed
...

Benefits:

- Easier retries
- Partial success is preserved
- Failures are isolated
- Progress can be shown clearly

If one image fails, the user does not lose the other eleven.

---

11. Processing Status

Each image/job should move through:

QUEUED
 ↓
PROCESSING
 ↓
COMPLETED

or:

QUEUED
 ↓
PROCESSING
 ↓
FAILED
 ↓
RETRY

The frontend can show:

> Processing 8 of 12 images

This is much better than making the user wait on a single loading screen without knowing what is happening.

---

12. Retry Logic

A failed request should not automatically become a permanent failure.

The backend can retry limited transient failures.

For example:

Attempt 1 → API/network failure
↓
Retry
↓
Attempt 2 → Failure
↓
Retry
↓
Attempt 3 → Failure
↓
Mark Failed

The exact retry count can be configured.

The system should not retry indefinitely.

---

13. Manual Review Is the Final Quality Gate

AI's job is to reduce manual work.

It does not eliminate verification.

The intended experience is:

BOOK IMAGE
↓
AI DOES 90% OF STRUCTURING WORK
↓
USER QUICKLY CHECKS
↓
FIXES ANY ERRORS
↓
APPROVES

The user should never need to type all 50 questions manually if extraction works properly.

---

14. AI Performance Insights

The application first calculates analytics using deterministic code.

Example:

Topic: Partnership Accounts

Accuracy: 38%
Recent Accuracy: 42%
Questions Attempted: 26
Average Time: 78 seconds
Confidence: Weak
Attention Level: High

Only then may this data be sent to OpenAI for interpretation.

The AI could return:

> Your performance in Partnership Accounts remains weak across recent attempts. Accuracy is low and you are spending more time than average on these questions. Prioritize this topic in upcoming revision sessions.

The AI is explaining existing facts.

It is not deciding the official numbers.

---

15. AI Insight Constraints

Insights should be:

- Short
- Specific
- Based only on supplied data
- Non-inventive
- Action-oriented

The AI should not claim:

> "You have mastered this topic"

unless the application data strongly supports it.

Likewise, it should not invent reasons such as:

> "You don't understand formulas"

unless there is actual data supporting that conclusion.

A safe pattern is:

OBSERVED DATA
↓
AI INTERPRETATION
↓
PRACTICAL SUGGESTION

---

16. What AI Does NOT Control

This boundary is important.

OpenAI does **not** control:

Revision scheduling

The scheduler is deterministic.

Test timer

Controlled by application timestamps.

Answer saving

Controlled by the backend/database.

Scoring

Calculated by application code.

Accuracy

Calculated by application code.

Negative marking

Calculated by application code.

Final weak-topic classification

Primarily calculated using defined rules.

Database permissions

Controlled by application security.

Authentication

Controlled by the authentication system.

AI assists; the application remains in control.

---

17. API Cost Control

Since this is a personal preparation application, unnecessary API usage should be avoided.

Cost-saving rules

Rule 1

Only process newly uploaded images.

Do not reprocess completed images without user action.

Rule 2

Store extraction results.

Never call OpenAI again merely to display an already processed question.

Rule 3

Use AI insights only when useful.

Do not generate an insight every time the dashboard loads.

For example, generate/update insights after:

- A completed mock test
- A meaningful analytics change
- An explicit user request

Rule 4

Process only relevant content.

Avoid repeatedly sending large historical datasets when a concise calculated summary is sufficient.

---

18. AI Request Security

The OpenAI API key must:

- Exist only in secure backend environment variables.
- Never be placed in frontend code.
- Never be exposed to the browser.
- Never be stored in the database as ordinary application data.
- Never be returned through an API response.

The request architecture is:

BROWSER
↓
YOUR BACKEND
↓
OPENAI API

Never:

BROWSER
↓
OPENAI API KEY EXPOSED

---

19. Data Privacy

Uploaded question images may contain copyrighted study material.

For V1, the system should:

- Process only user-uploaded material.
- Avoid publicly exposing uploaded images.
- Keep storage private.
- Restrict access to the owning user.
- Send content to AI only for the requested processing purpose.

The application should not create a public repository of uploaded books or question pages.

---

20. Failure Handling

OpenAI unavailable

The application keeps the image and processing job.

Status:

> Processing Failed — Retry Available

Invalid AI response

The backend does not save invalid content directly as approved questions.

It either:

- retries where appropriate, or
- flags the processing result for review.

Partial extraction

If the AI successfully extracts three questions but a fourth is unclear:

The three valid questions can be saved as drafts.

The unclear content can be flagged.

Partial success should not be discarded unnecessarily.

---

21. Future AI Possibilities — Explicitly Not V1

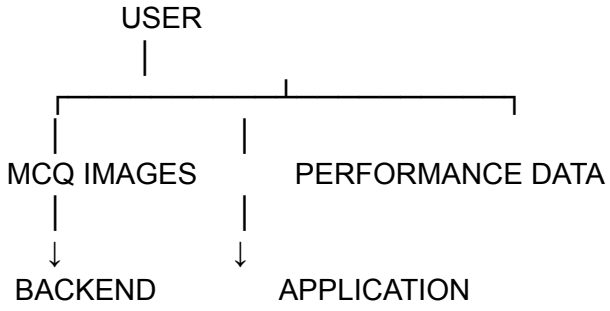
These ideas are intentionally outside the current scope:

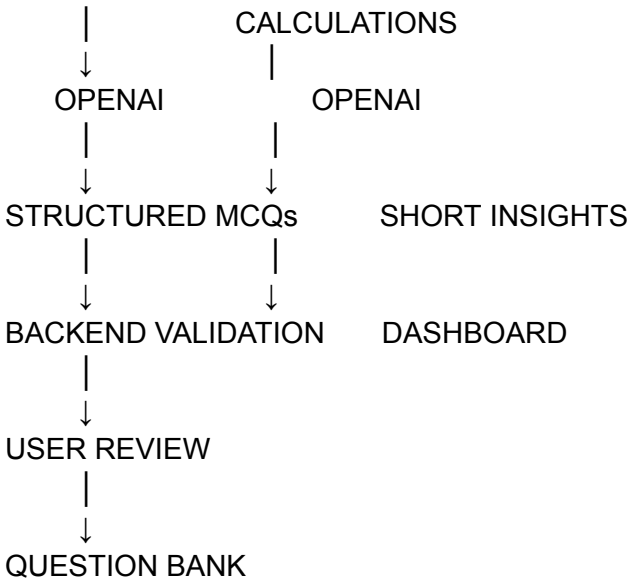
- Conversational AI tutor
- Ask doubts through chat
- AI-generated full study notes
- Automatic question generation
- Autonomous answer verification without review
- Voice tutor
- AI-generated video lessons
- Fully AI-controlled revision schedules

They may be considered later, but are **not part of the one-month MVP**.

---

22. V1 AI Architecture Summary





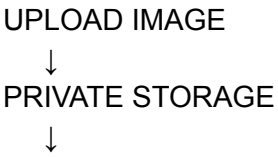
---

23. Core AI Rules

These are fixed for V1:

1. AI processes unstructured content; application code handles deterministic operations.
2. AI output must be structured.
3. AI output must be validated before storage.
4. AI extraction initially creates draft/reviewable questions.
5. AI must not silently invent missing question text.
6. AI must not silently invent correct answers.
7. Topic classification uses the application's existing topic list.
8. Images are processed independently for reliability.
9. Failed processing can be retried.
10. AI results are stored and reused.
11. API keys remain server-side.
12. AI-generated insights explain calculated data rather than replacing calculations.
13. API calls should be made only where they provide real value.

Final AI Flow



CREATE PROCESSING JOB
↓
OPENAI API
↓
STRUCTURED RESPONSE
↓
VALIDATE
↓
DRAFT QUESTIONS
↓
USER REVIEW
↓
APPROVED QUESTIONS
↓
QUESTION BANK
↓
MOCK TEST
↓
DETERMINISTIC RESULTS
↓
CALCULATED ANALYTICS
↓
OPTIONAL AI INSIGHT

DOCUMENT 06 STATUS: OPENAI API & AI ARCHITECTURE LOCKED

Next is **DOCUMENT 07 — UI/UX & Complete Screen Specification**, where we'll define the exact screens, navigation, layout priorities, actions, states, and user flows for the entire application—without changing the scope we've already locked.

DOCUMENT 07

DOCUMENT 07

UI/UX & Complete Screen Specification

1. UI/UX Goal

The application is used repeatedly during a stressful one-month preparation period. The interface should therefore be:

- Simple
- Fast
- Calm
- Focused
- Mobile-friendly, while working well on desktop
- Data-rich only where needed

The primary principle is:

> **The user should always know what to do next.**

No unnecessary screens, complex navigation, or feature clutter.

---

2. Main Navigation

The primary application navigation:

Dashboard
Revision
Mock Tests
Question Bank
Analytics
Settings

The two main working areas remain clear:

REVISION → What should I study?
MOCK TEST → How am I performing?

---

3. Screen 01 — Login

Purpose

Allow secure entry into the application.

Content

- Application logo/name
- Email authentication
- Login action

After login

- New user → Onboarding
- Existing user → Dashboard

No additional complexity is required.

---

4. Screen 02 — Exam Setup

Purpose

Create the preparation context.

Fields

- Exam Name
- Exam Date

Action

Continue

The selected exam date becomes the scheduler's deadline.

---

5. Screen 03 — Study Availability

Purpose

Tell the scheduler when the user can study.

The user selects available slots.

Example:

MONDAY
7:00 PM — 10:00 PM

TUESDAY
7:00 PM — 10:00 PM

SATURDAY
10:00 AM — 1:00 PM
6:00 PM — 9:00 PM

Actions

- Add Slot
- Edit Slot
- Delete Slot
- Continue

The screen should clearly display:

Total weekly study availability

---

6. Screen 04 — Add Syllabus

Purpose

Build the complete hierarchy:

Subject
↓
Chapter
↓

Topic

Example:

Accounting
├── Financial Accounting
│ ├── Journal Entries
│ ├── Ledger
│ └── Trial Balance
│
└── Final Accounts
├── Depreciation
└── Balance Sheet

Topic fields

- Topic name
- Estimated revision time
- Difficulty
- Importance

Actions

- Add Subject
- Add Chapter
- Add Topic
- Edit
- Delete

The user should be able to quickly add multiple items without unnecessary page reloads.

Primary action

Generate Revision Plan

---

7. Screen 05 — Initial Plan Review

Purpose

Show the initial plan after generation.

The user sees:

- Days remaining
- Total available study time
- Topics added
- Initial revision workload

Example:

27 Days Remaining

68 Topics
94 Hours Available

Your revision plan is ready.

Then:

Go to Dashboard

This is an overview—not a complex month-long editable timetable.

The detailed daily plan lives inside Revision.

---

8. Screen 06 — Main Dashboard

This is the application's home screen.

It should immediately answer:

> What do I need to do now?

Section A — Exam Countdown

27 DAYS LEFT
Uttarakhand Accountant Examination

Section B — Today's Revision

Example:

TODAY'S PLAN

✓ Journal Entries
Revision 3 · 30 min

● Partnership Accounts
Revision 2 · 45 min

○ Depreciation
Revision 3 · 30 min

○ Trial Balance
Quick Revision · 20 min

Primary action:

View Today's Plan

Section C — Progress

54 / 68
Topics Revised

3 / 4
Today's Tasks

Section D — Mock Test Snapshot

LAST MOCK TEST

76% Accuracy
Score: 38 / 50

Needs Attention:
Partnership Accounts

Action:

Take Mock Test

Section E — High Attention

A short list of priority areas.

Example:

HIGH ATTENTION

1. Partnership Accounts
2. Depreciation
3. Cost Accounting

Action:

View Analytics

---

9. Screen 07 — Today's Revision Plan

Purpose

Provide the complete actionable study list for the current day.

Each task shows:

- Topic name
- Subject/chapter context
- Revision number
- Estimated duration
- Priority/attention where useful
- Status

Example:

PARTNERSHIP ACCOUNTS
Accounting · Final Accounts

Revision 3
45 Minutes

[Start Revision]

When started

The task can enter:

In Progress

The app does not need to provide learning content.

She studies from her own material.

Complete Revision

After finishing:

Mark Complete

Optional quick confidence input:

How do you feel about this topic?

Strong
Okay
Weak

Then the system confirms completion and updates future scheduling.

---

10. Screen 08 — Revision Overview

Purpose

Show the entire syllabus and current preparation state.

Hierarchy:

ACCOUNTING

Financial Accounting
✓ Journal Entries
✓ Ledger
● Trial Balance

Final Accounts

- Depreciation
  - Partnership Accounts

Each topic can show:

- Revision count
- Last revised
- Next scheduled revision
- Attention level

Selecting a topic opens Topic Detail.

---

11. Screen 09 — Topic Detail

Purpose

Show everything relevant to one topic.

Example:

Depreciation

Revision Status
3 revisions completed

Last Revised
September 10

Next Revision
September 16

Confidence
Weak

Mock Performance
54% Accuracy

Attention
High

Revision History

Sep 02 — Revision 1 — Okay
Sep 06 — Revision 2 — Weak
Sep 10 — Revision 3 — Weak

This screen connects revision and performance data without making the user search through multiple screens.

---

12. Screen 10 — Schedule

Purpose

View upcoming revision work.

The default view should be simple:

TODAY
4 tasks

TOMORROW
3 tasks

SEP 08
4 tasks

A calendar-style view can be supported, but the important information is:

- Upcoming tasks
- Overdue tasks
- Estimated workload

The user is not expected to manually rebuild the schedule.

The scheduler controls planning.

---

13. Screen 11 — Revision History

Purpose

View completed revision activity.

Each record:

- Date
- Topic
- Revision number
- Confidence

Example:

Sep 10
Depreciation
Revision 3
Weak

This is mainly a tracking screen and should remain simple.

---

14. Screen 12 — Question Set List

Purpose

Manage groups of uploaded MCQs.

Example:

Accounting Practice Set 01
48 Questions · 46 Approved

[Continue Review]

Accounting Practice Set 02
50 Questions · Ready

Primary action:

Create Question Set

---

15. Screen 13 — Upload Questions

Purpose

Upload source images containing MCQs.

Fields:

- Question Set Name
- Optional subject/topic association
- Image uploader

The user can upload multiple images.

Example:

12 images selected

[Upload & Process]

After upload:

Processing begins.

---

16. Screen 14 — Processing Status

Purpose

Show progress without blocking the user.

Example:

PROCESSING QUESTIONS

8 / 12 Images Completed

✓ Page 01
✓ Page 02
✓ Page 03
● Page 04 Processing
✕ Page 05 Failed

Actions where needed:

- Retry Failed

- Review Extracted Questions

The user should be able to leave and return later.

---

17. Screen 15 — Review Extracted Questions

Purpose

Validate AI extraction.

Layout:

Source image reference

Available when required.

Editable question

Question 12

What is depreciation?

A. ...
B. ...
C. ...
D. ...

Correct Answer: B
Topic: Depreciation

Actions:

- Edit
- Delete
- Change Topic
- Change Correct Answer
- Approve

The screen should make reviewing many questions quick.

---

18. Screen 16 — Question Bank

Purpose

View all approved questions.

Filters:

- Question Set
- Subject
- Chapter
- Topic

Each item shows a short question preview and its topic.

Actions:

- View
- Edit
- Delete/Archive
- Select for Test

Primary action:

Create Mock Test

---

19. Screen 17 — Create Mock Test

Purpose

Select questions and define the test.

Fields:

- Test Name
- Question source/filter
- Number of questions
- Overall duration
- Marks per correct answer

- Negative marking

Example:

Accounting Mock Test 01

50 Questions
60 Minutes

+1 Correct
-0.25 Incorrect

Action:

Create Test

The application validates the test before allowing it to start.

---

20. Screen 18 — Test Instructions

Purpose

Provide a final summary before timing begins.

Example:

ACCOUNTING MOCK TEST 01

50 Questions
60 Minutes

Correct: +1
Incorrect: -0.25

[START TEST]

Important:

> The timer begins only after clicking Start Test.

---

21. Screen 19 — Test Runner

This is the most focused screen in the application.

Top area:

Question 11 of 50 42:16

Main area:

Which of the following...

○ A. Option
○ B. Option
○ C. Option
○ D. Option

Bottom actions:

[Previous] [Mark for Review] [Save & Next]

Additional action:

Question Navigator

Submit Test

The interface should avoid unrelated navigation.

---

22. Screen 20 — Question Navigator

Purpose

Allow quick movement across the test.

Example:

1 ✓ 2 ✓ 3 ✓ 4 ○ 5 ★
6 ✓ 7 ✓ 8 ● 9 ○ 10 ✓

Legend:

- ✓ Answered
- ○ Unanswered
- ★ Marked for Review
- ● Current

The candidate taps a number to jump to that question.

---

23. Screen 21 — Submit Confirmation

Before manual submission:

READY TO SUBMIT?

Answered: 43
Unanswered: 4
Marked for Review: 3

[Go Back] [Submit Test]

The user may still return to review questions.

Once confirmed, the attempt becomes final.

---

24. Screen 22 — Test Result

Purpose

Give an immediate, understandable result.

Example:

MOCK TEST COMPLETE

Score
36 / 50

Accuracy
82.6%

Correct 38
Incorrect 8
Skipped 4

Time Used
47m 23s

Actions:

- Review Questions
- Topic Performance
- Test History
- Dashboard

---

25. Screen 23 — Question Review

Purpose

Review test mistakes.

Each question shows:

- Question
- Selected answer
- Correct answer
- Correct/incorrect/skipped
- Time spent

The user can filter:

- All
- Incorrect
- Skipped
- Correct

This is where detailed mistake review happens.

---

26. Screen 24 — Test History

Purpose

Access previous attempts.

Example:

Mock Test 03
Sep 12
38 / 50 · 79%

Mock Test 02
Sep 08
40 / 50 · 83%

Mock Test 01
Sep 05
36 / 50 · 76%

Selecting an attempt opens its saved result.

---

27. Screen 25 — Analytics

Purpose

Provide the complete performance picture.

Overall

- Recent accuracy
- Overall trend
- Recent score
- Time performance

Topic Performance

Example:

Partnership Accounts
38% · High Attention

Depreciation
54% · Needs Attention

Journal Entries
90% · Strong

Revision Progress

- Topics revised
- Never revised
- Due
- Overdue

Performance Trend

Simple historical progression.

The goal is useful interpretation—not excessive graphs.

---

28. Screen 26 — Settings

Purpose

Manage preparation configuration.

Sections:

Exam

- Edit name
- Edit exam date

Study Availability

- Add/edit/remove slots

Account

- Basic account management

Settings should not contain core daily actions.

---

29. Important Empty States

The application must guide the user when data does not yet exist.

No topics

> Add your syllabus topics to generate a revision plan.

No revision today

> No revisions are currently scheduled for today.

The scheduler should then naturally show the next upcoming work.

No mock tests

> Upload MCQ questions to create your first mock test.

No analytics data

> Complete a mock test to start tracking performance.

No question-bank questions

> Upload images containing MCQs to begin.

---

30. Important Error States

Image processing failure

> This image could not be processed. Retry processing.

Question missing answer

Set a correct answer before using this question in a scored test.

No study availability

Add study availability before generating a revision plan.

Test unavailable

If configuration is incomplete, explain exactly what needs correction.

Errors should always provide a next action.

---

31. Responsive Behaviour

The application should be designed primarily for responsive web use.

Desktop

Best for:

- Adding syllabus
- Reviewing many extracted questions
- Question-bank management
- Analytics

Mobile

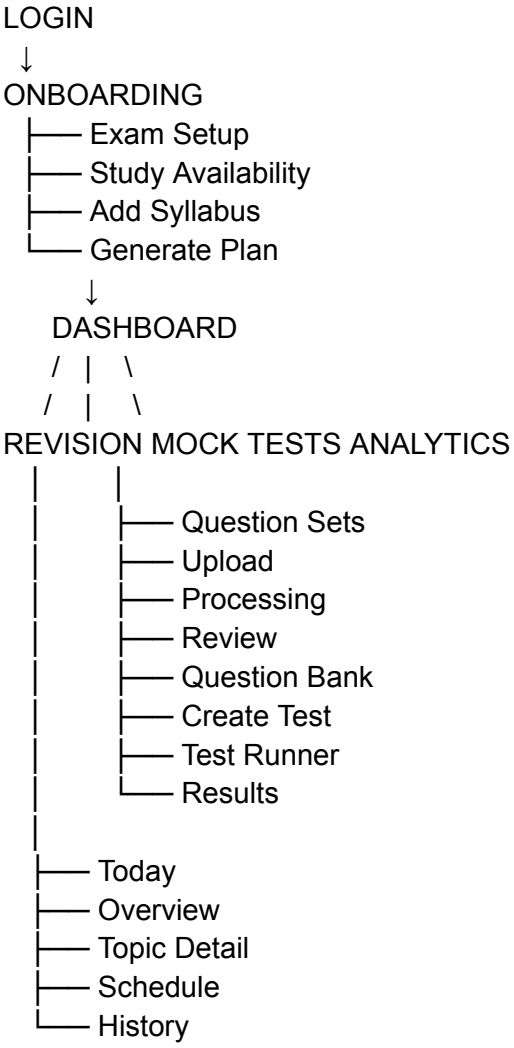
Best for:

- Viewing today's revision plan
- Marking revisions complete
- Checking progress
- Taking mock tests
- Reviewing quick results

The same product logic remains identical across devices.

---

32. Complete Navigation Flow



---

33. Core UX Rules

These remain fixed for V1:

1. Dashboard prioritizes today's action.
2. Revision screens focus on execution, not complex planning.
3. The scheduler manages complexity in the background.
4. AI extraction always goes through review before approval.
5. Mock testing uses a distraction-free interface.
6. The timer is always clearly visible during tests.
7. Answers are saved continuously.
8. Results are understandable immediately.
9. Detailed analytics remain separate from the main dashboard.
10. Every empty/error state should guide the user toward the next action.

11. Mobile usability is essential.
12. No screen should introduce features outside the locked scope.

---

34. Final UX Journey

OPEN APP
↓
SEE TODAY'S PLAN
↓
REVISE TOPICS
↓
MARK COMPLETE
↓
TAKE MOCK TEST
↓
VIEW RESULT
↓
IDENTIFY WEAK TOPICS
↓
SCHEDULER ADAPTS
↓
OPEN APP NEXT DAY
↓
SEE UPDATED PLAN

This keeps the entire application centered around one continuous preparation loop:

Plan → Revise → Test → Analyse → Adapt

DOCUMENT 07 STATUS: UI/UX & COMPLETE SCREEN SPECIFICATION LOCKED

Next is **DOCUMENT 08 — Technical Architecture & Stack**, where we lock the practical implementation architecture for building this quickly and reliably: frontend, backend, PostgreSQL, authentication, storage, OpenAI integration, background processing, security, and deployment.

DOCUMENT 08

DOCUMENT 08

Technical Architecture & Stack

1. Purpose

This document locks the practical technical foundation for building the application quickly within the available one-month preparation window.

The priority is:

> **Simple architecture, fast development, reliable behaviour, and no unnecessary infrastructure.**

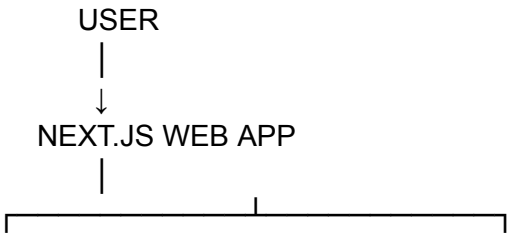
The system should support everything already defined:

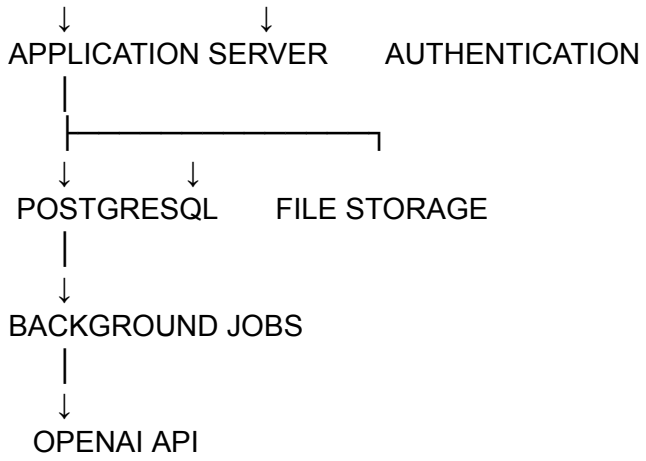
- Revision Scheduler
- Syllabus management
- Question image upload
- OpenAI-powered MCQ extraction
- Question review and approval
- Question bank
- Mock test runner
- Timers and answer persistence
- Analytics
- Adaptive revision priorities

---

2. Recommended Architecture

For V1, use a modern full-stack web architecture.





The key principle:

> **One primary application, one primary database, managed supporting services.**

Do not split this MVP into microservices.

---

3. Recommended Stack

Frontend + Application Layer

Next.js

Recommended for the main application.

Use it for:

- Frontend UI
- Server-side logic
- API endpoints/server actions where appropriate
- Authentication integration
- Responsive application shell

Why:

- Fast development
- One codebase
- Strong ecosystem

- Easy deployment
- Excellent fit for dashboard-style applications

Language

TypeScript

Use TypeScript across the project.

Why:

- Better data consistency
- Safer API integration
- Shared types
- Easier maintenance
- Useful for complex entities such as tests and revision tasks

---

Styling

Tailwind CSS

Use Tailwind for UI development.

Recommended because the product needs:

- Fast implementation
- Responsive layouts
- Consistent spacing
- Reusable components
- Dashboard and form-heavy interfaces

Do not spend unnecessary development time building a custom CSS architecture.

UI Components

Use a reliable component layer such as:

shadcn/ui

Useful for:

- Dialogs
- Forms
- Dropdowns
- Tabs
- Buttons
- Cards
- Tables
- Date pickers
- Alerts

This should accelerate development while allowing the product to maintain its own visual identity.

---

4. Database

PostgreSQL

PostgreSQL is the primary database.

It stores:

USERS
EXAMS
STUDY AVAILABILITY
SYLLABUS
TOPICS
REVISION TASKS
REVISION HISTORY
QUESTION DATA
MOCK TESTS
TEST ATTEMPTS
ANSWERS
ANALYTICS PROFILES

It is the system of record for application data.

---

5. ORM

Use:

Prisma ORM

Prisma will manage interaction between the application and PostgreSQL.

Responsibilities:

- Database schema
- Migrations
- Typed database access
- Relationships
- Safer queries

Architecture:

NEXT.JS
↓
PRISMA
↓
POSTGRESQL

The database schema should directly implement the entities locked in Document 05.

---

6. Authentication

For V1, authentication should remain simple.

Recommended:

Auth.js

Possible login methods:

- Email/password
- Magic link

For this application, there is no need to introduce multiple social login providers unless required.

The core requirement is:

Every user's preparation data remains isolated from every other user.

All important queries must be scoped to the authenticated user.

---

7. File Storage

Question images should not be stored directly inside PostgreSQL.

Use managed object/file storage.

Recommended options include:

- Cloudflare R2
- AWS S3
- Supabase Storage

For the simplest MVP, choose one storage provider and keep all uploaded question images private.

Architecture:

USER UPLOADS IMAGE
↓
APPLICATION VALIDATES
↓
PRIVATE OBJECT STORAGE
↓
DATABASE STORES FILE REFERENCE

The database stores:

- File path/key
- Original filename
- MIME type
- Processing state

---

8. Image Upload Flow

FRONTEND

↓
SELECT IMAGES
↓
VALIDATE TYPE + SIZE
↓
UPLOAD
↓
PRIVATE STORAGE
↓
CREATE DATABASE RECORD
↓
CREATE PROCESSING JOB
↓
BACKGROUND PROCESSING

The user should not need to keep the browser open until processing finishes.

---

9. Background Processing

AI image extraction should run outside the normal user request cycle.

Use a background job system.

Recommended V1 options:

Option A — Inngest

Very suitable for event-driven background workflows.

Option B — Trigger.dev

Also suitable for reliable background jobs.

The recommended direction for this MVP:

Next.js + Inngest

because the workflow is event-based and relatively straightforward.

Example:

IMAGE UPLOADED

↓
EVENT CREATED
↓
BACKGROUND JOB STARTS
↓
CALL OPENAI
↓
VALIDATE RESPONSE
↓
SAVE DRAFT QUESTIONS
↓
MARK JOB COMPLETE

---

10. Why Background Jobs Are Necessary

Without background processing:

USER WAITS
↓
REQUEST STAYS OPEN
↓
AI PROCESSES IMAGE
↓
RISK OF TIMEOUT

With background jobs:

USER UPLOADS
↓
JOB QUEUED
↓
USER CONTINUES USING APP
↓
WORKER PROCESSES IMAGE
↓
RESULT SAVED

This is more reliable and gives a better user experience.

---

11. OpenAI Integration Architecture

The API key exists only on the server.

BROWSER
  │
  │ No OpenAI key
  ↓
NEXT.JS SERVER
  │
  │ Secure API key
  ↓
OPENAI API

For MCQ extraction:

SOURCE IMAGE
  ↓
BACKGROUND JOB
  ↓
OPENAI API
  ↓
STRUCTURED OUTPUT
  ↓
VALIDATION
  ↓
DATABASE

The application should use structured output/schema enforcement supported by the selected OpenAI API at implementation time.

The AI response should then pass through application-level validation before being saved.

---

12. Validation Layer

Use:

Zod

Zod validates data at application boundaries.

Use it for:

- Forms
- API inputs
- AI responses
- Test configuration
- Question structures

Example architecture:

USER INPUT
↓
ZOD VALIDATION
↓
APPLICATION LOGIC
↓
DATABASE

And:

OPENAI RESPONSE
↓
ZOD VALIDATION
↓
BUSINESS VALIDATION
↓
DATABASE

This is especially important because AI output must never be trusted blindly.

---

13. Revision Scheduler Architecture

The scheduler should be ordinary deterministic application code.

It does not require AI.

Conceptually:

TOPICS
+
EXAM DATE
+

STUDY AVAILABILITY
+
REVISION HISTORY
+
PRIORITY SIGNALS
↓
SCHEDULER ENGINE
↓
REVISION TASKS

The scheduler can be implemented as a service/module.

For example:

```
/lib
  /scheduler
    calculatePriority.ts
    calculateDueRevision.ts
    generatePlan.ts
    rebalanceSchedule.ts
```

The exact filenames can vary, but the logic should remain isolated from UI code.

---

14. Scheduler Execution

The scheduler is triggered when meaningful events occur.

Examples:

Initial syllabus setup

TOPICS CREATED
↓
GENERATE INITIAL PLAN

Revision completed

REVISION COMPLETED
↓
UPDATE HISTORY
↓
CALCULATE NEXT REVISION
↓

UPDATE FUTURE TASKS

Mock test completed

TEST SUBMITTED
↓
CALCULATE PERFORMANCE
↓
UPDATE TOPIC PROFILE
↓
UPDATE PRIORITY
↓
REEVALUATE FUTURE REVISION TASKS

The scheduler should never run continuously without reason.

---

15. Mock Test Architecture

The mock test runner has two parts:

Frontend

Responsible for:

- Displaying questions
- Selecting answers
- Navigation
- Question states
- Visual timer
- Autosaving interactions

Backend

Responsible for:

- Attempt creation
- Official end timestamp
- Answer persistence
- Final submission
- Score calculation
- Auto-submission rules

Architecture:

START TEST
↓
SERVER CREATES ATTEMPT
↓
ENDS_AT TIMESTAMP STORED
↓
FRONTEND RUNS DISPLAY TIMER
↓
ANSWERS AUTOSAVE
↓
USER SUBMITS / TIME EXPIRES
↓
SERVER CALCULATES RESULT

The backend timestamp remains authoritative.

The browser timer is only the display mechanism.

---

16. Answer Autosave

Answers should not depend on a final submit button.

Recommended flow:

USER SELECTS OPTION
↓
LOCAL UI UPDATES
↓
SAVE REQUEST
↓
DATABASE UPDATED

For reliability, save operations can be debounced where appropriate.

The application should also save:

- Answer changes
- Mark-for-review state
- Time tracking checkpoints where required

A refresh should not normally destroy the entire attempt.

---

17. Timer Reliability

When an attempt begins:

started_at = 10:00
ends_at = 11:00

If the page refreshes at:

10:24

the frontend calculates remaining time from `ends_at`.

It does not restart at 60 minutes.

The backend validates submission time.

This prevents simple timer manipulation through refreshes.

---

18. Scoring Service

Scoring should be isolated into deterministic logic.

Example:

attemptAnswers
+
correctAnswers
+
marksPerCorrect
+
negativeMarks
↓
SCORING SERVICE
↓
FINAL RESULT

Suggested structure:

```
/lib
  /testing
    calculateScore.ts
    calculateAccuracy.ts
    finalizeAttempt.ts
```

OpenAI is not involved.

---

19. Analytics Architecture

Analytics should be generated from actual stored events.

```
REVISION HISTORY
      +
TEST ATTEMPTS
      +
ATTEMPT ANSWERS
      ↓
ANALYTICS SERVICE
      ↓
TOPIC PERFORMANCE
      ↓
DASHBOARD DATA
```

Suggested logical separation:

```
/lib
  /analytics
    calculateTopicPerformance.ts
    calculateTrend.ts
    determineAttentionLevel.ts
    generateSummary.ts
```

Analytics profiles may be cached in the database after calculation.

However:

> Historical revisions and test answers remain the source of truth.

---

20. API Architecture

Keep APIs organized around domain actions.

Example:

```
/api
  /exams
  /topics
  /availability
  /revision
  /question-sets
  /questions
  /uploads
  /mock-tests
  /attempts
  /analytics
```

The exact Next.js route structure may differ, but domain separation should remain clear.

Avoid one giant endpoint handling unrelated operations.

---

21. Application Layer Structure

Recommended project structure:

```
/app
  /(auth)
  /(dashboard)
    /dashboard
    /revision
    /mock-tests
    /question-bank
    /analytics
    /settings

/components
  /ui
  /dashboard
  /revision
  /tests
```

```
  /questions
  /analytics

/lib
  /scheduler
  /testing
  /analytics
  /ai
  /validation

/server
  /services
  /repositories

/prisma
  schema.prisma
```

The exact structure can evolve slightly during implementation.

The important architectural rule is:

> **UI, business logic, database access, and AI integration should not become one mixed layer.**

---

22. Suggested Service Boundaries

Scheduler Service

Controls:

- Initial plan generation
- Revision intervals
- Priority scoring
- Capacity allocation
- Rebalancing

Question Processing Service

Controls:

- Upload metadata
- Job creation
- AI extraction workflow
- Validation
- Draft question creation

Test Service

Controls:

- Test creation
- Attempt creation
- Answer persistence
- Submission
- Scoring

Analytics Service

Controls:

- Topic performance
- Accuracy
- Speed metrics
- Trends
- Attention levels

AI Service

Controls:

- OpenAI requests
- Prompt construction
- Structured response handling
- AI insight generation

The AI service should not directly modify unrelated application entities without validation through the appropriate domain service.

---

23. Security Rules

Authentication

Every protected request must identify the authenticated user.

Authorization

A user must access only their own:

- Exams
- Topics
- Images
- Questions
- Mock tests
- Attempts
- Analytics

Never trust a user ID sent directly from the frontend without authorization checks.

API Key Protection

Environment variables should contain:

- Database credentials
- OpenAI API key
- Storage credentials
- Authentication secrets

None should be exposed through client-side code.

Upload Validation

Validate:

- Allowed image types
- File size
- Number of files
- Ownership

Reject unsupported or suspicious uploads.

---

24. Error Handling

Every major service should return meaningful errors.

Example:

AI extraction

Unable to process this image.
The image has been saved and you can retry.

Test submission

This test has already been submitted.

Invalid question

A correct answer is required before this question can be added to a scored test.

Errors should not expose internal database, provider, or API details to the user.

---

25. Logging

For V1, log important backend events.

Examples:

- AI processing failure
- Background job failure

- Test finalization error
- Database error
- Upload failure

Recommended approach:

Use structured server logs and an error-monitoring service such as:

Sentry

This is useful because errors can be diagnosed quickly during the short preparation period.

---

26. Deployment

A simple recommended deployment architecture:

Application

Vercel

Database

Managed PostgreSQL provider

Storage

Cloudflare R2 / Supabase Storage / S3

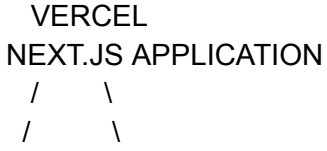
Background Jobs

Inngest

Error Monitoring

Sentry

Architecture:



This minimizes server management.

---

27. Recommended Final Stack

For this specific MVP:

Layer	Technology
Framework	Next.js
Language	TypeScript
Styling	Tailwind CSS
Components	shadcn/ui
Database	PostgreSQL
ORM	Prisma
Authentication	Auth.js
Validation	Zod
File Storage	Cloudflare R2 or Supabase Storage
Background Jobs	Inngest
AI	OpenAI API
Error Monitoring	Sentry
Deployment	Vercel

This is a practical stack—not an unnecessarily complicated one.

---

28. Environment Variables

Conceptually:

DATABASE_URL=

AUTH_SECRET=

OPENAI_API_KEY=

STORAGE_ACCESS_KEY=
STORAGE_SECRET_KEY=
STORAGE_BUCKET=
STORAGE_REGION=

INNGEST_EVENT_KEY=
INNGEST_SIGNING_KEY=

SENTRY_DSN=

Actual variable names may differ depending on providers.

These must never be committed to the public repository.

---

29. Build Order

Technical implementation should follow dependency order.

1. PROJECT FOUNDATION
   ↓
2. DATABASE + AUTH
   ↓
3. EXAM + AVAILABILITY
   ↓
4. SYLLABUS MANAGEMENT
   ↓

5. REVISION SCHEDULER
↓
6. REVISION EXECUTION + HISTORY
↓
7. IMAGE UPLOAD + STORAGE
↓
8. OPENAI EXTRACTION
↓
9. QUESTION REVIEW + BANK
↓
10. MOCK TEST CREATION
↓
11. TEST RUNNER
↓
12. RESULTS + SCORING
↓
13. ANALYTICS
↓
14. ADAPTIVE SCHEDULING
↓
15. FINAL TESTING + DEPLOYMENT

This order avoids building screens that depend on systems that do not yet exist.

---

30. What We Are Deliberately Avoiding

For this one-month MVP, do not introduce:

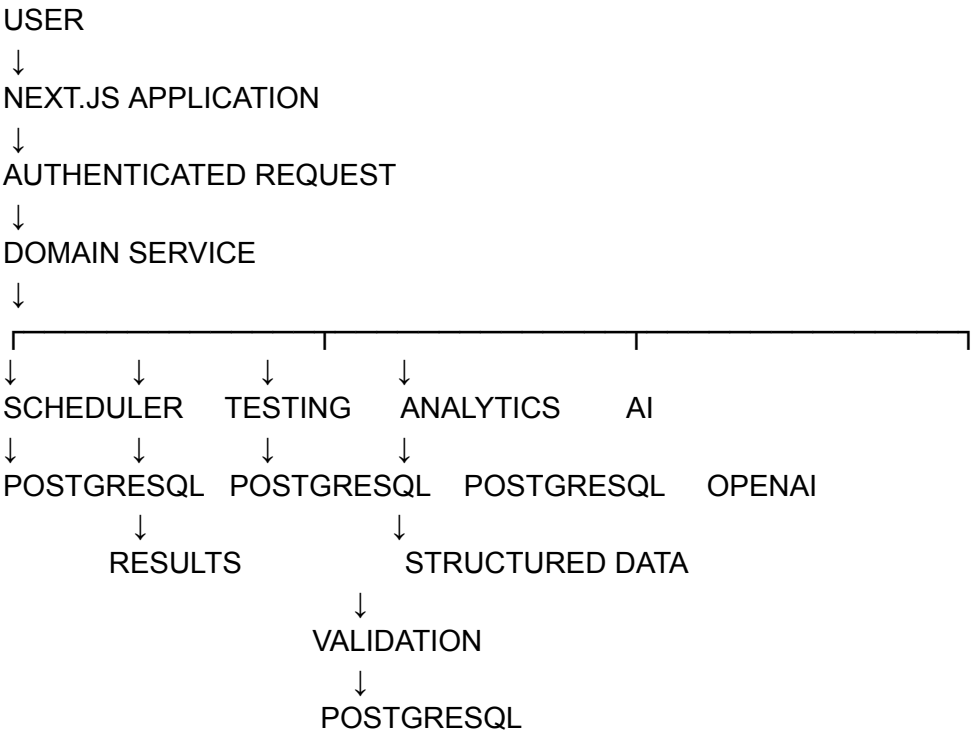
- Microservices
- Kubernetes
- Multiple databases
- Complex event buses
- Redis unless a real requirement emerges
- Custom authentication infrastructure
- AI agents
- Real-time collaboration
- Native mobile apps
- Separate frontend/backend repositories

The architecture should remain:

One focused web application with clearly separated internal modules.

---

31. Complete Technical Flow



---

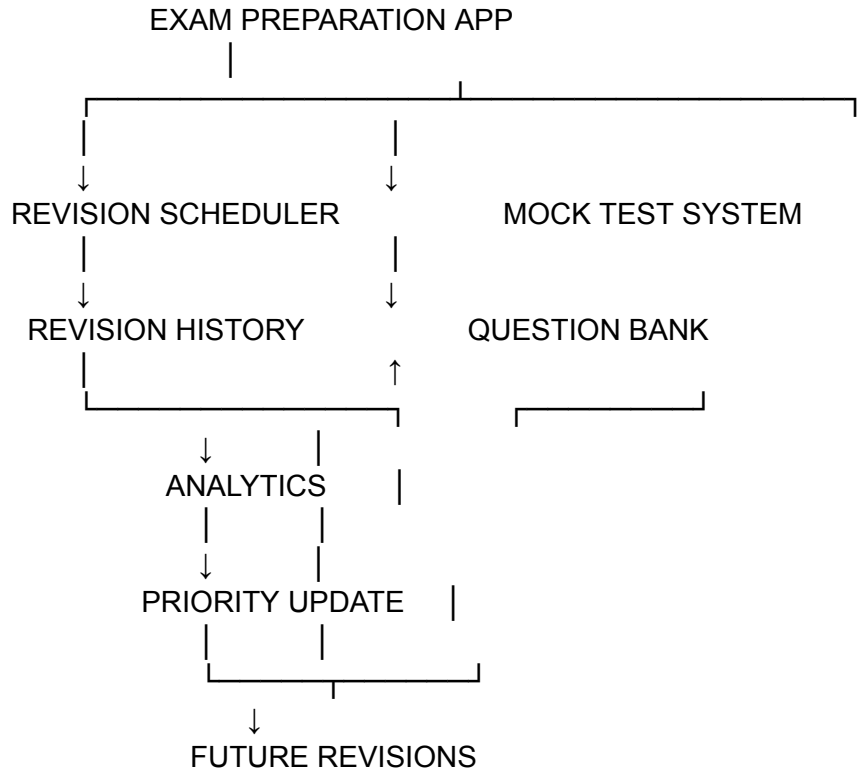
32. Core Technical Rules

These are now fixed for V1:

1. Build one Next.js application.
2. Use TypeScript throughout.
3. PostgreSQL is the primary database.
4. Prisma manages database access.
5. OpenAI is server-side only.
6. AI image processing runs through background jobs.
7. All AI output is validated before database storage.
8. The scheduler remains deterministic application logic.
9. Scoring remains deterministic application logic.
10. Test timing is controlled by server timestamps.
11. Answers are autosaved.

12. Historical attempts use snapshots.
13. Images are stored in private object storage.
14. Authentication and authorization protect every user's data.
15. Domain logic remains separated from UI components.
16. Deployment uses managed services wherever possible.
17. No unnecessary infrastructure for V1.

Final Architecture Summary



MCQ IMAGES
↓
PRIVATE STORAGE
↓
BACKGROUND JOB
↓
OPENAI API
↓
VALIDATED STRUCTURED DATA
↓
QUESTION REVIEW

↓
QUESTION BANK

DOCUMENT 08 STATUS: TECHNICAL ARCHITECTURE & STACK LOCKED

Next is **DOCUMENT 09 — Complete Development Roadmap & Build Plan**, where we turn everything already locked into the exact implementation sequence: **what to build first, module-by-module deliverables, dependencies, testing checkpoints, and the fastest path to getting a usable working application without losing scope or direction.**

DOCUMENT 09

DOCUMENT 09

Complete Development Roadmap & Build Plan

1. Purpose

This document converts all previously locked product decisions into a **practical build sequence**.

The objective is not to build everything simultaneously.

The objective is:

> **Build the application in the correct dependency order so that a usable product is available as quickly as possible.**

The scope remains exactly what we have already defined:

PLAN
↓
REVISE
↓
TRACK
↓
UPLOAD MCQs
↓
PROCESS
↓
CREATE TEST
↓
TAKE TEST
↓
ANALYSE
↓
ADAPT

No new product modules are introduced here.

---

2. Development Strategy

The build should happen in **five implementation stages**.

STAGE 1
Foundation + Data Setup
↓
STAGE 2
Revision System
↓
STAGE 3
Question Processing System
↓
STAGE 4
Mock Test System
↓
STAGE 5
Analytics + Adaptation + Finalization

Each stage should produce something functional.

Do not wait until the end to discover whether the system works.

---

3. Stage 1 — Foundation & Core Setup

Objective

Create the technical base on which every other module depends.

Step 1.1 — Create Application

Set up:

- Next.js
- TypeScript

- Tailwind CSS
- shadcn/ui
- Basic application layout

Initial structure:

/app
/components
/lib
/server
/prisma

At this stage, the application does not need final visual polish.

The goal is a stable foundation.

Step 1.2 — Configure Database

Set up:

- PostgreSQL
- Prisma
- Database connection
- Initial migrations

Create the initial core models:

User
Exam
StudyAvailability
Subject
Chapter
Topic

Verify:

Application
↓
Prisma
↓
PostgreSQL

Before proceeding, basic create/read/update/delete operations should work.

Step 1.3 — Authentication

Implement:

- Login
- Logout
- Protected application routes
- User ownership checks

Verify that:

> User A cannot access User B's data.

Even if the first user is currently the only intended candidate, ownership architecture should work correctly from the beginning.

Step 1.4 — Application Shell

Build the main navigation:

Dashboard
Revision
Mock Tests
Question Bank
Analytics
Settings

Some sections can initially display empty states.

The purpose is to establish the application's structure early.

Stage 1 Deliverable

At completion:

✓ User can log in
✓ Protected app exists
✓ Database works

✓ Exam can be created
✓ Basic app navigation exists
✓ User data is isolated

---

4. Stage 2 — Revision System

This is the first complete core product module.

Objective

Allow the candidate to:

1. Enter the syllabus
2. Define study availability
3. Generate a revision plan
4. Complete revisions
5. Track revision history

Step 2.1 — Exam Setup

Build:

Exam Setup Screen

Fields:

- Exam name
- Exam date

Validation:

- Required name
- Valid future exam date

Output:

EXAM CONTEXT CREATED

Step 2.2 — Study Availability

Build:

- Weekly availability form
- Multiple slots per day
- Add slot
- Edit slot
- Delete slot

Example:

Monday
7 PM–10 PM

Wednesday
6 PM–9 PM

Saturday
10 AM–1 PM
6 PM–9 PM

Calculate total weekly availability.

Step 2.3 — Syllabus Management

Build the hierarchy:

SUBJECT
↓
CHAPTER
↓
TOPIC

Each topic supports:

- Name
- Estimated revision minutes
- Difficulty
- Importance

Required actions:

Add
Edit
Delete
Reorder if needed

At this point, the candidate should be able to enter the entire syllabus.

Step 2.4 — Revision Scheduler

Implement the scheduler engine.

Inputs:

Exam Date
Study Availability
Topics
Estimated Revision Duration
Difficulty
Importance
Revision History
Priority Signals

Initial output:

REVISION TASKS

Each task contains:

- Topic
- Revision number
- Scheduled date
- Estimated duration
- Priority
- Status

The initial scheduler should first ensure the core revision cycle works correctly.

Do not overcomplicate optimization before basic scheduling is reliable.

Step 2.5 — Daily Revision Experience

Build:

Dashboard Today's Plan

Shows:

- Today's topics
- Revision number
- Estimated duration
- Completion status

Revision Execution

The candidate:

STARTS
↓
STUDIES USING OWN MATERIAL
↓
MARKS COMPLETE
↓
SELECTS CONFIDENCE

Confidence:

- Strong
- Okay
- Weak

Step 2.6 — Revision History

On completion:

REVISION TASK
↓
STATUS UPDATED
↓
REVISION HISTORY CREATED
↓
NEXT REVISION LOGIC UPDATED

Build:

- Revision overview
- Topic detail
- Revision history

Stage 2 Deliverable

The application should now work independently as a revision tool:

✓ Exam configured
✓ Study time configured
✓ Full syllabus added
✓ Revision plan generated
✓ Daily tasks visible
✓ Tasks completed
✓ Confidence recorded
✓ History preserved
✓ Future revisions updated

At this point, the first major product loop is usable:

Plan → Revise → Track

---

5. Stage 3 — Question Processing System

Objective

Turn uploaded MCQ images into an approved question bank.

Step 3.1 — File Storage

Configure private storage.

Implement:

Upload Image
↓

Validate
↓
Private Storage
↓
Database Reference

Validate:

- File type
- File size
- Ownership

Create:

- Question Set
- Source Image records

Step 3.2 — Processing Jobs

Set up Inngest.

Implement the job lifecycle:

QUEUED
↓
PROCESSING
↓
COMPLETED

or:

QUEUED
↓
PROCESSING
↓
FAILED
↓
RETRY

The application must support processing images independently.

Step 3.3 — OpenAI MCQ Extraction

Implement:

SOURCE IMAGE
↓
BACKGROUND JOB
↓
OPENAI
↓
STRUCTURED RESPONSE

Extract:

- Question text
- Options
- Visible correct answer, if available
- Topic suggestion
- Extraction status

The backend then validates the response.

Step 3.4 — Draft Question Creation

Successful extraction creates draft questions.

AI RESPONSE
↓
VALIDATE
↓
QUESTIONS
↓
STATUS: NEEDS REVIEW

Do not automatically treat extracted questions as approved.

Step 3.5 — Question Review Interface

Build the review workflow.

The user can:

- Edit question text
- Edit options
- Select/change correct answer
- Assign/change topic
- Approve
- Delete/archive

After approval:

APPROVED
↓
QUESTION BANK

Step 3.6 — Question Bank

Build:

- Question list
- Topic filter
- Subject filter
- Chapter filter
- Question-set filter

The question bank becomes the source for mock tests.

Stage 3 Deliverable

✓ Question sets created
✓ Images uploaded
✓ Images stored privately
✓ Background jobs process images
✓ OpenAI extracts MCQs
✓ Output validated
✓ Questions reviewed
✓ Questions approved
✓ Question bank functional

The second major loop is now available:

Upload → Extract → Review → Approve

---

6. Stage 4 — Mock Test System

Objective

Turn approved questions into a complete exam simulation.

Step 4.1 — Mock Test Creation

Build:

- Test name
- Question selection
- Question count
- Time limit
- Marks per correct answer
- Negative marking

Create:

MOCK TEST
↓
TEST QUESTION ORDER

Validate before test activation:

- Questions exist
- Correct answers exist
- Configuration is valid

Step 4.2 — Attempt Creation

When the candidate clicks:

Start Test

The backend creates:

- Test attempt
- `started_at`
- `ends_at`
- Attempt-question snapshots
- Initial answer records

This is the beginning of the official attempt.

Step 4.3 — Test Runner

Build:

- Current question display
- Options
- Previous
- Save & Next
- Mark for Review
- Question navigator
- Visible countdown
- Submit button

The test runner should remain distraction-free.

Step 4.4 — Answer Autosave

Every meaningful answer action should persist.

SELECT ANSWER
↓
UI UPDATES
↓
SAVE
↓
DATABASE

Also save:

- Answer changes
- Review status
- Question interaction state

A refresh should restore the attempt.

Step 4.5 — Timer Enforcement

The server-controlled `ends_at` timestamp remains authoritative.

ATTEMPT STARTS
↓
SERVER STORES ENDS_AT
↓
FRONTEND CALCULATES REMAINING TIME
↓
TIME EXPIRES
↓
FINALIZE ATTEMPT

Refreshing the browser must not restart the timer.

Step 4.6 — Submission & Scoring

On submission:

LOCK ATTEMPT
↓
CALCULATE ANSWERS
↓
CALCULATE SCORE
↓
CALCULATE ACCURACY
↓
SAVE RESULT

Calculate:

- Correct
- Incorrect
- Skipped
- Attempted
- Score
- Accuracy

- Time used

All calculations remain deterministic.

Step 4.7 — Results & Review

Build:

Results Screen

Shows:

- Score
- Accuracy
- Correct
- Incorrect
- Skipped
- Time used

Question Review

Shows:

- Question
- Selected answer
- Correct answer
- Result
- Time spent

Test History

Shows every completed attempt.

Stage 4 Deliverable

✓ Mock tests created
✓ Questions assigned
✓ Attempt starts correctly
✓ Timer works
✓ Answers autosave

✓ Refresh recovery works
✓ Manual submission works
✓ Expiry works
✓ Scores calculate correctly
✓ Results saved
✓ Historical attempts preserved

The third major loop is now complete:

Create → Attempt → Score → Review

---

7. Stage 5 — Analytics & Adaptive Scheduling

Objective

Connect revision behaviour and mock-test performance into one preparation intelligence loop.

Step 5.1 — Topic Performance Calculation

After each completed attempt:

ATTEMPT ANSWERS
↓
GROUP BY TOPIC
↓
CALCULATE PERFORMANCE

Calculate:

- Questions attempted
- Correct
- Incorrect
- Skipped
- Accuracy
- Recent accuracy
- Average time

- Trend

Step 5.2 — Attention Levels

Apply the rules from Document 04.

Output:

- High Attention
- Needs Attention
- Stable
- Strong
- Limited Data

Do not classify a topic aggressively when data is insufficient.

Step 5.3 — Performance Profiles

Update:

topic_performance_profiles

This provides fast access for:

- Dashboard
- Analytics
- Scheduler

Historical attempt data remains the source of truth.

Step 5.4 — Analytics Screens

Build:

Overall Performance

- Recent score
- Accuracy

- Time usage
- Trend

Topic Performance

- Accuracy
- Attention level
- Trend

Weak Areas

Prioritized list.

Revision Progress

- Revised topics
- Never revised
- Due
- Overdue

Test History

Historical performance.

Step 5.5 — Adaptive Scheduler

Connect analytics to revision priorities.

MOCK TEST RESULT
↓
TOPIC PERFORMANCE
↓
ATTENTION LEVEL
↓
PRIORITY SCORE UPDATE
↓
SCHEDULER REEVALUATES
↓
FUTURE REVISION TASKS ADJUST

Critical rule:

> Do not rebuild the entire schedule after every test.

Only future planning is intelligently adjusted.

Step 5.6 — AI Insights

Finally add optional AI-generated explanations.

Input:

CALCULATED DATA ONLY

Output:

SHORT PREPARATION INSIGHT

Example:

> Partnership Accounts remains your highest-attention topic based on repeated low accuracy and slower answer times.

Generate these only:

- After meaningful test completion
- After meaningful performance changes
- When explicitly requested

Do not call the API unnecessarily.

Stage 5 Deliverable

✓ Topic performance calculated
✓ Attention levels assigned
✓ Trends tracked
✓ Revision progress tracked
✓ Analytics dashboard complete
✓ Weak topics identified
✓ Revision priority updated
✓ Future schedule adapts
✓ Optional AI insights work

The complete product loop is now functional:

Plan → Revise → Test → Analyse → Adapt

---

8. Final Integration Phase

After all five stages, test the full end-to-end journey.

Test Scenario 1 — New User

CREATE ACCOUNT
↓
CREATE EXAM
↓
ADD AVAILABILITY
↓
ADD SYLLABUS
↓
GENERATE PLAN
↓
COMPLETE REVISION

Verify every transition.

Test Scenario 2 — Question Upload

CREATE QUESTION SET
↓
UPLOAD MULTIPLE IMAGES
↓
PROCESS
↓
ONE IMAGE FAILS
↓
RETRY
↓
REVIEW QUESTIONS
↓
APPROVE

Verify partial failures do not affect completed images.

Test Scenario 3 — Mock Test

CREATE TEST
↓
START
↓
ANSWER QUESTIONS
↓
MARK SOME FOR REVIEW
↓
REFRESH PAGE
↓
RESTORE ATTEMPT
↓
CONTINUE
↓
SUBMIT
↓
VERIFY RESULT

Test Scenario 4 — Timer

START TEST
↓
WAIT / REFRESH
↓
VERIFY TIME DOES NOT RESET
↓
ALLOW TIME TO EXPIRE
↓
VERIFY ATTEMPT FINALIZATION

Test Scenario 5 — Adaptive Learning

COMPLETE REVISION
↓
TAKE MOCK TEST
↓

PERFORM POORLY IN TOPIC
↓
VERIFY ANALYTICS UPDATE
↓
VERIFY ATTENTION LEVEL
↓
VERIFY FUTURE REVISION PRIORITY

This scenario validates the application's central intelligence loop.

---

9. Development Priority

If time becomes limited, development priority is:

Priority 1 — Must Work

Revision Scheduler
Revision Completion
Revision History

Question Upload
AI Extraction
Question Review

Mock Test
Timer
Autosave
Scoring

Basic Analytics

Priority 2 — Important

Topic Performance Profiles
Attention Levels
Adaptive Scheduling
Detailed Analytics
Test History

Priority 3 — Polish

AI Performance Insights
Advanced visualizations
Additional UI refinements
Extra convenience improvements

This priority order protects the application's core purpose.

---

10. Recommended One-Month Build Schedule

Since the exam is only about one month away, the application itself should reach usability quickly.

Week 1 — Foundation + Revision

Day 1–2
Project + Database + Auth

Day 3
Exam + Availability

Day 4–5
Syllabus Management

Day 6–7
Scheduler + Daily Revision

End of Week 1:

> A usable revision planner exists.

Week 2 — MCQ Processing

Day 8
Storage + Upload

Day 9
Question Sets + Processing Jobs

Day 10–11
OpenAI Extraction

Day 12–13
Question Review

Day 14
Question Bank

End of Week 2:

> Book images can become approved questions.

Week 3 — Mock Tests

Day 15–16
Test Creation

Day 17–19
Test Runner + Autosave

Day 20
Timer + Submission

Day 21
Results + Review + History

End of Week 3:

> The complete mock-test experience works.

Week 4 — Intelligence + Stabilization

Day 22–23
Analytics

Day 24

Topic Performance

Day 25
Adaptive Scheduling

Day 26
AI Insights

Day 27–28
End-to-End Testing

Day 29
Bug Fixes

Day 30
Final Deployment + Stability Check

---

11. Critical Development Rule

Do not work like this:

Build 20 screens
↓
Hope backend works later

Instead:

BUILD MODULE
↓
CONNECT DATA
↓
TEST MODULE
↓
VERIFY COMPLETE FLOW
↓
MOVE FORWARD

Each stage must work before unnecessary complexity is added.

---

12. Recommended Development Checkpoints

There are five major checkpoints.

CHECKPOINT 01

Can a user log in and create an exam?

CHECKPOINT 02

Can the user enter the syllabus and receive a working revision plan?

CHECKPOINT 03

Can uploaded MCQ images become approved questions?

CHECKPOINT 04

Can the user take a complete timed mock test reliably?

CHECKPOINT 05

Does mock-test performance actually influence future revision priority?

If all five answers are **yes**, the MVP is fundamentally complete.

---

13. Final Product Definition

By the end of development, the candidate should be able to do this:

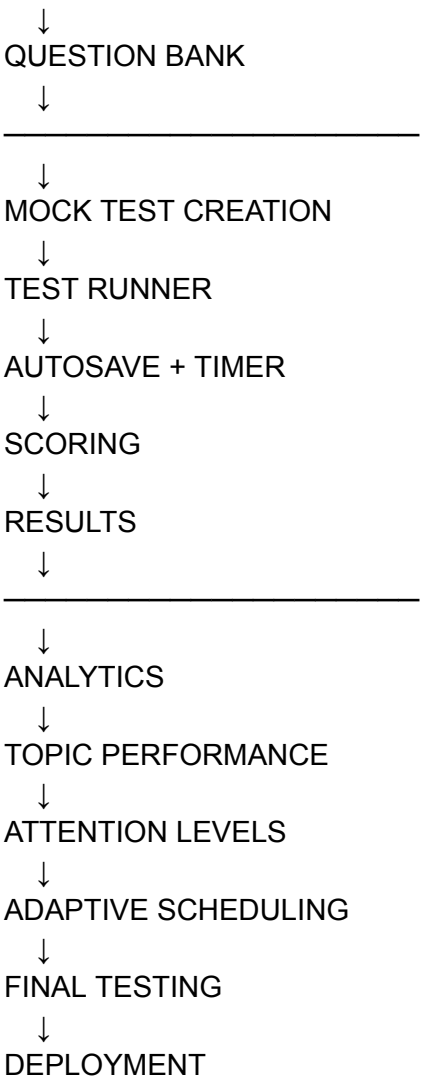
1. OPEN APPLICATION
   ↓
2. SEE DAYS LEFT FOR EXAM
   ↓
3. SEE WHAT TO REVISE TODAY
   ↓
4. COMPLETE REVISION
   ↓
5. RECORD CONFIDENCE
   ↓
6. UPLOAD MCQ BOOK PAGES

↓
7. AI EXTRACTS QUESTIONS
↓
8. REVIEW & APPROVE QUESTIONS
↓
9. CREATE / TAKE MOCK TEST
↓
10. RECEIVE RESULTS
↓
11. REVIEW MISTAKES
↓
12. SEE WEAK TOPICS
↓
13. SYSTEM ADJUSTS FUTURE REVISION PRIORITIES
↓
14. REPEAT UNTIL EXAM

---

14. Complete Development Flow

FOUNDATION
↓
AUTH + DATABASE
↓
EXAM SETUP
↓
AVAILABILITY
↓
SYLLABUS
↓
REVISION SCHEDULER
↓
REVISION TRACKING
↓
————————————————
↓
IMAGE UPLOAD
↓
BACKGROUND JOBS
↓
OPENAI EXTRACTION
↓
QUESTION REVIEW



---

15. Final Locked Build Principles

These principles govern the implementation:

1. Build the core preparation workflow first.
2. Do not expand the product scope.
3. Each module must work end-to-end before moving forward.
4. Revision scheduling is the first core system.
5. AI extraction is a processing assistant, not autonomous truth.
6. Question review remains the quality gate.
7. Mock-test timing and scoring are deterministic.
8. Historical preparation data is preserved.
9. Analytics comes from actual recorded behaviour.

10. Adaptation adjusts future work intelligently, not chaotically.
11. Infrastructure remains simple.
12. The application must become useful before it becomes polished.
13. The goal is a reliable exam-preparation tool within the available preparation period.

DOCUMENT 09 STATUS: COMPLETE DEVELOPMENT ROADMAP & BUILD PLAN LOCKED

At this point, the **product scope, functionality, data architecture, AI architecture, UI/UX, technical stack, and implementation sequence are all locked**.

The next logical document is **DOCUMENT 10 — Final MVP Specification & Developer Handoff**, combining the entire project into one concise master reference containing the exact product definition, modules, rules, architecture, and build requirements for implementation.

DOCUMENT 10

DOCUMENT 10

Final MVP Specification & Developer Handoff

1. Project Definition

Product

A focused **exam preparation web application** for an Uttarakhand government Accountant-related competitive examination.

Primary User

Initially, one accountant aspirant preparing for an exam approximately one month away.

Core Problem

The candidate needs to manage two major parts of preparation:

1. **Revision**
    - What to revise
    - When to revise it
    - Whether revision is actually happening
    - Which topics need more attention
2. **Mock Tests**
    - Convert MCQs from book images into digital questions
    - Create timed mock tests
    - Track answers and performance
    - Identify weak topics

Core Product Loop

PLAN
↓
REVISE
↓
TRACK
↓

TEST
↓
ANALYSE
↓
ADAPT
↓
REPEAT

The application exists to make this loop easier and more systematic.

---

2. Primary Product Principle

The application should always help answer:

> **What should I do next to prepare better for the exam?**

The user should not need to manually maintain a complicated revision timetable or calculate performance across multiple mock tests.

The system handles the tracking and scheduling logic in the background.

---

3. Final MVP Scope

The MVP contains exactly these major modules:

1. Authentication
2. Exam Setup
3. Study Availability
4. Syllabus Management
5. Revision Scheduler
6. Revision Tracking
7. Question Image Upload
8. AI MCQ Extraction
9. Question Review
10. Question Bank
11. Mock Test Creation
12. Mock Test Runner
13. Results & Question Review

14. Analytics
15. Adaptive Revision Priorities
16. Settings

No additional major modules are required for V1.

---

4. Module 1 — Authentication

The user can:

- Log in
- Log out
- Access protected application routes

System requirement:

> All user-owned data must remain isolated.

A user must never access another user's:

- Exam
- Topics
- Revision history
- Images
- Questions
- Mock tests
- Test attempts
- Analytics

---

5. Module 2 — Exam Setup

The user configures:

- Exam name
- Exam date

The exam date is used as the preparation deadline.

The dashboard displays:

X DAYS LEFT

The scheduler uses the exam date to determine available preparation time.

---

6. Module 3 — Study Availability

The user manually defines available study periods.

Example:

Monday
7:00 PM – 10:00 PM

Tuesday
7:00 PM – 10:00 PM

Saturday
10:00 AM – 1:00 PM
6:00 PM – 9:00 PM

The system uses these slots as scheduling capacity.

The scheduler must not blindly schedule revision outside configured availability.

---

7. Module 4 — Syllabus Management

The syllabus hierarchy is:

SUBJECT
↓
CHAPTER
↓
TOPIC

Example:

Accounting
│
├── Financial Accounting

```
│   ├── Journal Entries
│   ├── Ledger
│   └── Trial Balance
│
└── Final Accounts
    ├── Depreciation
    └── Partnership Accounts
```

Each topic contains:

- Topic name
- Estimated revision duration
- Difficulty
- Importance

The user can:

- Add
- Edit
- Delete

The user controls the syllabus data.

---

8. Module 5 — Revision Scheduler

This is one of the application's core engines.

Inputs

Exam Date
+
Study Availability
+
Topics
+
Estimated Revision Duration
+
Difficulty
+
Importance
+
Revision History

+
Confidence
+
Mock Test Performance

Output

A set of revision tasks:

Topic
Revision Number
Scheduled Date
Estimated Duration
Priority
Status

Example:

Sep 05
Journal Entries
Revision 2
30 minutes

Sep 05
Depreciation
Revision 3
45 minutes

The scheduler should distribute work according to available capacity.

---

9. Revision Scheduling Rules

The scheduler is deterministic application logic.

It does **not** use AI to decide schedules.

Its responsibilities include:

- Generating the initial plan
- Determining future revisions
- Respecting available study time

- Tracking overdue work
- Prioritizing weak or important topics
- Updating future tasks after meaningful new information

The scheduler should adapt future planning.

It should not unnecessarily destroy or completely rebuild historical planning.

---

10. Module 6 — Revision Tracking

The daily dashboard shows today's revision tasks.

For every task, the user can:

START
↓
STUDY USING OWN MATERIAL
↓
MARK COMPLETE
↓
SELECT CONFIDENCE

Confidence options:

- Strong
- Okay
- Weak

On completion:

TASK COMPLETED
↓
REVISION HISTORY SAVED
↓
TOPIC UPDATED
↓
FUTURE REVISION RECALCULATED

The application tracks:

- Revision count
- Last revised
- Next revision

- Completion history
- Confidence

---

11. Module 7 — Question Image Upload

The user creates a question set and uploads images containing MCQs.

Example:

Accounting Practice Set 01

The system supports:

- Multiple image uploads
- File validation
- Private storage
- Independent processing
- Failed-image retry

The source image is preserved as a reference.

---

12. Module 8 — AI MCQ Extraction

The system uses the OpenAI API to process uploaded question images.

Flow:

IMAGE
↓
BACKGROUND JOB
↓
OPENAI API
↓
STRUCTURED EXTRACTION
↓
VALIDATION
↓
DRAFT QUESTIONS

The expected extraction structure includes:

Question Text
Options
Correct Answer
Topic Suggestion
Source Reference

Important rule:

> AI output is not automatically treated as correct.

Every extracted question enters review status.

---

13. Module 9 — Question Review

Before a question enters the active question bank, the user can:

- Edit question text
- Edit options
- Set/change correct answer
- Assign/change topic
- Approve
- Delete/archive

Lifecycle:

DRAFT
↓
NEEDS REVIEW
↓
APPROVED
↓
AVAILABLE FOR MOCK TESTS

Only approved questions should be used in scored mock tests.

---

14. Module 10 — Question Bank

The question bank stores approved MCQs.

Questions can be filtered by:

- Question set
- Subject
- Chapter
- Topic

The question bank supports:

- Viewing
- Editing
- Archiving/deleting
- Selecting questions for tests

It becomes the single source for mock-test questions.

---

15. Module 11 — Mock Test Creation

The user creates a test by configuring:

- Test name
- Questions
- Number of questions
- Overall duration
- Marks per correct answer
- Negative marking

Example:

Accounting Mock Test 01

50 Questions
60 Minutes

Correct: +1
Incorrect: -0.25

The system validates that:

- Questions exist
- Correct answers are available

- Configuration is valid

The test can then be started.

---

16. Module 12 — Mock Test Runner

When the user clicks **Start Test**:

The backend creates:

- Test attempt
- `started_at`
- `ends_at`
- Question snapshots
- Attempt answer records

The test interface provides:

- Current question
- Options
- Previous
- Save & Next
- Mark for Review
- Question navigator
- Visible countdown
- Submit button

Example:

Question 11 of 50 42:16

Which of the following...

○ A. Option
○ B. Option
○ C. Option
○ D. Option

[Previous] [Mark for Review] [Save & Next]

The test runner should remain distraction-free.

---

17. Timer Rules

The server controls official test timing.

Example:

started_at = 10:00
ends_at = 11:00

If the user refreshes at 10:24:

> Remaining time is calculated from `ends_at`.

The timer must never restart because of:

- Page refresh
- Browser reopening
- Navigation recovery

The backend timestamp remains authoritative.

---

18. Answer Autosave

Answers must be persisted during the test.

Flow:

USER SELECTS ANSWER
↓
UI UPDATES
↓
ANSWER SAVED
↓
DATABASE

Also save:

- Changed answers
- Mark-for-review status

- Relevant attempt state

A refresh should restore the active attempt.

The final submit button must not be the first time answers are saved.

---

19. Test Submission & Scoring

The test ends when:

1. User manually submits, or
2. Official time expires

On finalization:

LOCK ATTEMPT
↓
COMPARE ANSWERS
↓
CALCULATE SCORE
↓
CALCULATE ACCURACY
↓
SAVE RESULTS

Results include:

- Correct
- Incorrect
- Skipped
- Attempted
- Score
- Accuracy
- Time used

Scoring is deterministic.

OpenAI is not involved in scoring.

---

20. Module 13 — Results & Question Review

Immediately after completion, show:

MOCK TEST COMPLETE

Score
36 / 50

Accuracy
82.6%

Correct: 38
Incorrect: 8
Skipped: 4

Time Used: 47m 23s

The user can then:

- Review all questions
- Review incorrect questions
- Review skipped questions
- See selected answers
- See correct answers
- View topic performance
- Open test history

---

21. Module 14 — Analytics

Analytics is generated from actual recorded preparation behaviour.

Data sources:

REVISION HISTORY
+
TEST ATTEMPTS
+
ATTEMPT ANSWERS

The system calculates:

Revision Analytics

- Topics revised
- Never revised
- Due
- Overdue
- Revision frequency

Mock-Test Analytics

- Score
- Accuracy
- Correct
- Incorrect
- Skipped
- Time usage
- Historical trend

Topic Analytics

- Topic accuracy
- Recent accuracy
- Average answer time
- Trend
- Attention level

---

22. Attention Levels

Topics can be classified as:

HIGH ATTENTION
NEEDS ATTENTION
STABLE
STRONG
LIMITED DATA

Signals can include:

- Low accuracy

- Repeated mistakes
- Slow answering
- Weak revision confidence
- High topic importance

The system must avoid strong conclusions where data is insufficient.

---

23. Module 15 — Adaptive Revision Priorities

This connects testing with revision.

Flow:

MOCK TEST COMPLETED
↓
TOPIC PERFORMANCE CALCULATED
↓
ATTENTION LEVEL UPDATED
↓
TOPIC PRIORITY UPDATED
↓
FUTURE REVISION TASKS REEVALUATED

Example:

Depreciation
Low accuracy
+
Weak confidence
+
High importance
↓
Higher revision priority

Important:

> Future tasks are adjusted intelligently. The entire schedule is not randomly rebuilt after every test.

---

24. AI Role in the Final Product

OpenAI is used for:

Required V1 Role

MCQ IMAGE EXTRACTION

Optional later role:

SHORT PERFORMANCE INSIGHTS

Example:

> Partnership Accounts remains your highest-attention topic based on repeated low accuracy and slower answer times.

AI receives calculated performance data.

AI does not determine:

- Official scores
- Correct answers after approval
- Timer rules
- Revision completion
- Core scheduler logic

Those remain deterministic application functions.

---

25. Main User Journey

LOGIN
↓
SET EXAM DATE
↓
DEFINE STUDY AVAILABILITY
↓
ADD SYLLABUS
↓
GENERATE REVISION PLAN
↓
SEE TODAY'S TASKS

↓
COMPLETE REVISION
↓
UPLOAD MCQ IMAGES
↓
AI EXTRACTS QUESTIONS
↓
REVIEW & APPROVE
↓
CREATE MOCK TEST
↓
TAKE TEST
↓
GET RESULTS
↓
REVIEW MISTAKES
↓
ANALYTICS IDENTIFY WEAK TOPICS
↓
SCHEDULER ADJUSTS FUTURE PRIORITIES
↓
REPEAT

---

26. Primary Screens

The final V1 application contains:

01. Login
02. Exam Setup
03. Study Availability
04. Syllabus Setup
05. Initial Plan Review
06. Dashboard
07. Today's Revision
08. Revision Overview
09. Topic Detail
10. Schedule
11. Revision History
12. Question Set List
13. Upload Questions
14. Processing Status
15. Review Extracted Questions

16. Question Bank
17. Create Mock Test
18. Test Instructions
19. Test Runner
20. Question Navigator
21. Submit Confirmation
22. Test Results
23. Question Review
24. Test History
25. Analytics
26. Settings

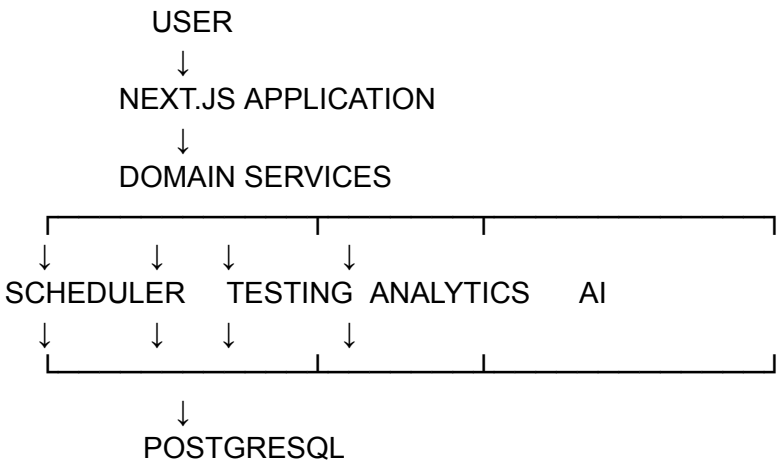
---

27. Technical Stack

Layer	Technology
Application	Next.js
Language	TypeScript
Styling	Tailwind CSS
Components	shadcn/ui
Database	PostgreSQL
ORM	Prisma
Authentication	Auth.js
Validation	Zod
File Storage	Managed private object storage
Background Jobs	Inngest
AI	OpenAI API
Error Monitoring	Sentry
Deployment	Vercel

---

28. Final Architecture



MCQ IMAGE
↓
PRIVATE STORAGE
↓
BACKGROUND JOB
↓
OPENAI API
↓
VALIDATED EXTRACTION
↓
QUESTION REVIEW
↓
QUESTION BANK

---

29. Core Technical Rules

1. One primary Next.js application.
2. TypeScript throughout.
3. PostgreSQL is the system of record.
4. Prisma manages database access.
5. OpenAI API keys remain server-side.
6. Image processing runs through background jobs.
7. AI output is validated before storage.
8. Scheduler logic is deterministic.
9. Scoring logic is deterministic.
10. Server timestamps control official test duration.

11. Answers autosave during tests.
12. Historical test attempts use question snapshots.
13. Uploaded images remain privately stored.
14. Every database operation enforces user ownership.
15. Business logic remains separate from UI components.
16. Use managed infrastructure where possible.
17. Avoid unnecessary microservices and infrastructure.

---

30. Recommended Build Sequence

PHASE 1
Foundation
- ├── Next.js
- ├── Database
- ├── Prisma
- ├── Authentication
- └── App Shell

PHASE 2
Revision System
- ├── Exam Setup
- ├── Availability
- ├── Syllabus
- ├── Scheduler
- ├── Daily Tasks
- └── Revision History

PHASE 3
Question Processing
- ├── Storage
- ├── Upload
- ├── Background Jobs
- ├── OpenAI Extraction
- ├── Validation
- ├── Review
- └── Question Bank

PHASE 4
Mock Tests
- ├── Test Creation
- ├── Attempt Creation

├── Test Runner
├── Autosave
├── Timer
├── Submission
├── Scoring
└── Results

PHASE 5
Intelligence
├── Analytics
├── Topic Performance
├── Attention Levels
├── Adaptive Scheduling
└── Optional AI Insights

PHASE 6
Finalization
├── End-to-End Testing
├── Bug Fixes
├── Security Checks
└── Deployment

---

31. Development Priorities

Must Work First

Revision Scheduler
Revision Completion
Revision History

Question Upload
AI Extraction
Question Review

Mock Test Runner
Timer
Autosave
Scoring

Basic Analytics

Important After Core Stability

Topic Performance
Attention Levels
Adaptive Scheduling
Detailed Analytics

Final Polish

AI Insights
Advanced Visualizations
Extra UI Refinements
Convenience Features

---

32. One-Month Development Target

Week 1

Foundation + Revision System

Week 2

Question Upload + OpenAI Extraction + Question Bank

Week 3

Mock Test System

Week 4

Analytics + Adaptive Scheduling + Testing + Deployment

The goal is that the product becomes useful early rather than waiting until the final days.

---

33. Explicitly Out of Scope for V1

Do **not** introduce:

- Native Android/iOS applications

- AI tutors or chatbots
- AI-generated teaching content
- Video lessons
- Social/community features
- Real-time collaboration
- Complex gamification
- Microservices
- Multiple databases
- Custom infrastructure
- AI-controlled scoring
- AI-controlled timer logic
- Automatic blind approval of extracted questions

These can only be considered after the core system is working.

---

34. MVP Completion Criteria

The MVP is complete when the candidate can successfully perform this entire journey:

CREATE ACCOUNT
↓
SET EXAM
↓
ADD AVAILABLE STUDY TIME
↓
ADD FULL SYLLABUS
↓
GENERATE REVISION PLAN
↓
COMPLETE DAILY REVISIONS
↓
VIEW REVISION HISTORY
↓
UPLOAD MCQ IMAGES
↓
AI EXTRACTS QUESTIONS
↓
REVIEW QUESTIONS
↓
APPROVE QUESTIONS
↓

CREATE MOCK TEST
↓
START TIMED ATTEMPT
↓
ANSWERS AUTOSAVE
↓
TIMER REMAINS CORRECT AFTER REFRESH
↓
SUBMIT OR AUTO-FINALIZE
↓
RECEIVE SCORE
↓
REVIEW MISTAKES
↓
VIEW TOPIC PERFORMANCE
↓
SYSTEM UPDATES FUTURE REVISION PRIORITIES

If this entire journey works reliably, the application has achieved its intended V1 purpose.

---

35. Final Product Statement

> **This is a focused adaptive exam-preparation system that helps an aspirant manage revision according to available study time, convert existing MCQ material into digital practice tests, track actual performance, identify weak topics, and continuously adjust future revision priorities until the examination.**

The complete product intelligence remains centered on one loop:

PLAN → REVISE → TEST → ANALYSE → ADAPT

DOCUMENT 10 STATUS: FINAL MVP SPECIFICATION & DEVELOPER HANDOFF LOCKED

The complete scope and direction are now consolidated. Development can proceed directly from these specifications without adding iterations or expanding the product beyond the defined MVP.

Tab 11